SENIOR BACKEND INTERVIEW PREPARATION

Interview questions, implementation exercises and system design

Go | Docker | CDN | Redis | Kafka | Databases | Trading Systems

An integrated study guide for senior backend engineering

How to use this guide

Use each section in three passes: explain core concepts aloud, implement the practical exercises, then connect the topic to production tradeoffs. For senior interviews, make assumptions explicit, discuss failure modes, and quantify bottlenecks where possible.

Contents

* Go, backend services and engineering fundamentals

* Infrastructure, caching and event-driven systems

* Database systems and data engineering

* HLD, LLD and trading platform system design

CDN, resilience, cloud, security, observability and leadership topics are integrated with the matching sections.

Go, backend services and engineering fundamentals

Complete Golang Interview Preparation - Questions + Practical Problems

For a Senior Backend Engineer (5+ years), I would prepare Go in three dimensions

Theory: Understand Go language features and internals.

Practical coding: Be able to implement common data structures, concurrency patterns and backend components.

Real-world scenarios: Understand how Go behaves in production, especially with Kafka, Redis, PostgreSQL and distributed services.

Below is a consolidated preparation list, including theoretical questions, coding problems and real-world backend scenarios.

1. Go fundamentals

Interview questions

What is Golang, and why was it designed?

What are the differences between Go and Java?

What are the differences between var, := and const?

What are zero values in Go?

What are pointers? Does Go support pointer arithmetic?

What is the difference between arrays and slices?

How does a slice work internally?

What are len() and cap()?

What happens when slice capacity is exceeded?

What is the difference between make() and new()?

What is the difference between nil slice and empty slice?

What are structs?

What are methods in Go?

What is the difference between value receiver and pointer receiver?

Does Go support method overloading?

What are variadic functions?

What are closures?

What is defer?

What is panic and recover?

What is the difference between break, continue and return?

What are exported and unexported identifiers?

What are Go packages and modules?

What is the difference between rune and byte?

How does string immutability work in Go?

What is the difference between shallow copy and deep copy?

Practical coding problems

No.	Problem	Level
1	Reverse a string	Easy
2	Check whether a string is a palindrome	Easy
3	Find duplicate elements in an array	Easy
4	Find the second-largest number	Easy
5	Count character frequency	Easy
6	Find missing number in an array	Easy
7	Remove duplicates from a slice	Easy
8	Merge two sorted slices	Medium
9	Rotate an array by K positions	Medium
10	Implement custom string splitting	Medium
11	Deep copy a nested struct	Medium
12	Implement a custom dynamic array	Hard

2. Slices, Maps and Internals

Interview questions

How are slices internally represented?

What is a slice header?

How does slice expansion work?

What happens when two slices share the same underlying array?

How can memory leaks happen through slices?

How do you copy a slice?

How does a Go map work internally?

Are maps thread-safe?

What happens when multiple goroutines write to the same map?

What happens when you read a nonexistent map key?

What happens when you delete a nonexistent map key?

Can you use a slice as a map key?

What types are allowed as map keys?

How does map iteration work?

How do you safely access maps concurrently?

Practical coding problems

No.	Problem	Level
1	Find duplicate values using a map	Easy
2	Count word frequency	Easy
3	Group employees by department	Medium
4	Find common elements between two slices	Medium
5	Implement a set using maps	Medium
6	Implement an LRU cache	Medium
7	Implement a thread-safe map	Medium
8	Implement an LFU cache	Hard
9	Build an in-memory key-value store	Hard

3. Interfaces and OOP concepts

Interview questions

What are interfaces in Go?

How does Go implement interfaces?

What is implicit interface implementation?

What is an empty interface?

What is any?

What is type assertion?

What is a type switch?

What is a typed nil interface problem?

Can an interface contain a nil pointer?

What is interface composition?

Can a struct implement multiple interfaces?

What is embedding?

Difference between embedding and inheritance?

How does polymorphism work in Go?

How do you implement dependency injection?

What are the advantages of small interfaces?

Practical coding problems

Design a payment interface supporting UPI, cards and net banking.

Implement a notification system using interfaces.

Implement a storage interface supporting Redis and PostgreSQL.

Build a logger interface with multiple implementations.

Implement a repository pattern.

Implement a factory pattern.

Implement a strategy pattern.

Implement a decorator pattern.

Design an extensible order-processing interface.

4. Goroutines and Concurrency

Very high priority

Golang Concurrency: From "Hello World" to Worker Pools - DEV Community

Interview questions

What is a goroutine?

How is a goroutine different from an OS thread?

How does Go schedule goroutines?

Explain the G-M-P scheduler model.

What is concurrency vs parallelism?

What are channels?

What is a buffered channel?

What is an unbuffered channel?

What happens when you send data to a closed channel?

What happens when you receive from a closed channel?

What happens when you send data to a nil channel?

What happens when you receive from a nil channel?

What is a deadlock?

What is a race condition?

What is the select statement?

What happens if multiple select cases are ready?

What is a sync.Mutex?

Difference between Mutex and RWMutex?

What is sync.WaitGroup?

What is sync.Once?

What is sync.Cond?

What is sync.Map?

What is atomic operation?

What is a goroutine leak?

How do you implement graceful goroutine shutdown?

What is the difference between worker pool and semaphore?

What is backpressure?

How do you limit concurrent goroutines?

How do you detect data races?

What is the difference between channel-based synchronization and mutex-based synchronization?

Practical coding problems

No.	Problem	Level
1	Print numbers using goroutines	Easy
2	Print odd and even numbers using two goroutines	Medium
3	Implement producer-consumer pattern	Medium
4	Implement worker pool	Medium
5	Implement bounded worker pool	Medium
6	Implement concurrent counter	Medium
7	Implement thread-safe bank account	Medium
8	Implement rate limiter	Medium
9	Implement semaphore using channels	Medium
10	Implement fan-in pattern	Medium
11	Implement fan-out pattern	Medium
12	Implement pipeline pattern	Medium
13	Implement concurrent HTTP requests with a limit	Medium
14	Implement a concurrent task scheduler	Hard
15	Implement graceful shutdown of workers	Hard
16	Find and fix goroutine leaks	Hard
17	Implement a concurrent job queue	Hard
18	Implement a bounded concurrent queue	Hard

5. Memory Management and Garbage Collection

Interview questions

What is stack memory and heap memory?

What is escape analysis?

How does Go decide whether an object goes on stack or heap?

What is garbage collection?

Explain Go's garbage collector.

What is the tri-color marking algorithm?

What is Stop-The-World (STW)?

What is concurrent garbage collection?

What is the role of GOGC?

What is GOMEMLIMIT?

What is memory fragmentation?

What is memory allocation?

What is sync.Pool?

When should you use object pooling?

How do you identify memory leaks?

How do you reduce allocations in Go?

What is the difference between new() and object pooling?

How do you analyze heap profiles?

What is the difference between CPU profiling and memory profiling?

How does garbage collection affect latency-sensitive applications?

Practical problems

Write a program to compare stack and heap allocations.

Create a program that demonstrates escape analysis.

Implement an object pool using sync.Pool.

Optimize a program that creates unnecessary allocations.

Identify and fix a memory leak caused by a slice.

Identify and fix a goroutine-related memory leak.

Profile a Go application using pprof.

Optimize a high-allocation JSON processing application.

Compare performance with and without sync.Pool.

Build a memory-intensive application and analyze GC behavior.

6. Error Handling, Panic and Context

Interview questions

How does error handling work in Go?

Why does Go return errors instead of using exceptions?

What is the difference between errors.New() and fmt.Errorf()?

What is error wrapping?

What is errors.Is()?

What is errors.As()?

What is the difference between panic and error?

When should you use recover()?

Where should recover() be placed?

What happens if a panic occurs inside a goroutine?

What is context.Context?

Why is context important in backend applications?

Difference between context.Background() and context.TODO()?

Explain context.WithCancel().

Explain context.WithTimeout().

Explain context.WithDeadline().

What is context.WithValue()?

Why should context be passed as the first parameter?

Why should you not store context inside a struct?

How do you propagate cancellation across microservices?

Practical problems

Implement custom error types.

Implement error wrapping across service and repository layers.

Implement a function that returns multiple error types.

Implement a timeout-based HTTP request.

Implement a cancellable worker pool.

Implement a request that stops when its context expires.

Implement graceful cancellation of multiple goroutines.

Implement a service with timeout propagation across multiple dependencies.

Build a retry mechanism that respects context cancellation.

Implement a panic recovery middleware in Gin.

7. Go Data Structures and Algorithms

Interview questions

What is the difference between an array, slice and linked list?

How do stacks work?

How do queues work?

What is a binary search tree?

What is a heap?

What is a hash table?

What is collision handling?

What is a trie?

What is a graph?

What is BFS and DFS?

What is time complexity?

What is space complexity?

What is recursion?

What is dynamic programming?

What is a sliding window algorithm?

Practical coding problems

No.	Problem	Level
1	Two Sum	Easy
2	Valid Parentheses	Easy
3	Binary Search	Easy
4	Reverse Linked List	Easy
5	Merge Two Sorted Lists	Easy
6	Maximum Subarray	Medium
7	Longest Substring Without Repeating Characters	Medium
8	Product of Array Except Self	Medium
9	3Sum	Medium
10	Merge Intervals	Medium
11	Search in Rotated Sorted Array	Medium
12	Top K Frequent Elements	Medium
13	Kth Largest Element	Medium
14	Implement Min Stack	Medium
15	Detect Cycle in Linked List	Medium
16	Binary Tree Level Order Traversal	Medium
17	Lowest Common Ancestor	Medium
18	Number of Islands	Medium
19	LRU Cache	Medium
20	Median From Data Stream	Hard
21	Word Ladder	Hard
22	Serialize and Deserialize Binary Tree	Hard

8. Go Backend Development - HTTP, REST and Gin

Interview questions

How does an HTTP server work in Go?

What is net/http?

What is the difference between http.Handle() and http.HandleFunc()?

How does http.ListenAndServe() work?

What is the Gin framework?

What is middleware?

How do you implement custom middleware in Gin?

How does routing work in Gin?

How do you validate request payloads?

How do you implement authentication and authorization?

How do you implement JWT authentication?

How do you handle CORS?

How do you handle HTTP timeouts?

How do you implement graceful HTTP server shutdown?

How do you manage HTTP connection pooling?

What is the difference between HTTP/1.1, HTTP/2 and HTTP/3?

What is keep-alive?

How do you handle request body size limits?

How do you implement API versioning?

How do you implement request tracing and correlation IDs?

Practical problems

Build a basic REST API using net/http.

Build CRUD APIs using Gin.

Implement JWT authentication middleware.

Implement request validation middleware.

Implement a centralized error-handling middleware.

Implement rate-limiting middleware.

Implement request logging middleware.

Implement correlation ID middleware.

Implement API pagination and filtering.

Implement graceful server shutdown.

Build an API supporting concurrent requests.

Implement an API timeout mechanism.

Build a file upload and download API.

Implement API idempotency using an idempotency key.

9. Database and PostgreSQL

Interview questions

How do you connect Go with PostgreSQL?

What is database/sql?

What is the difference between sql.DB and sql.Tx?

What is connection pooling?

What is SetMaxOpenConns()?

What is SetMaxIdleConns()?

What is SetConnMaxLifetime()?

What is the N+1 query problem?

How do you prevent SQL injection?

What are prepared statements?

How do transactions work?

Explain ACID properties.

What are isolation levels?

What is deadlock in a database?

What is optimistic locking?

What is pessimistic locking?

What are indexes?

What is the difference between B-Tree and Hash indexes?

What is database replication?

What is connection exhaustion?

How do you handle database failures?

How do you implement retry logic for transactions?

How do you handle concurrent updates to the same order?

What is the difference between sql.NullString and a pointer?

How do you optimize slow queries?

Practical problems

Build CRUD APIs using PostgreSQL.

Implement database connection pooling.

Implement a transaction transferring money between two accounts.

Implement concurrent-safe order updates.

Implement optimistic locking using a version column.

Implement a repository pattern with PostgreSQL.

Implement pagination using LIMIT and OFFSET.

Implement cursor-based pagination.

Implement bulk inserts.

Implement batch processing.

Implement retry logic for failed transactions.

Handle duplicate requests using unique constraints.

Build a database migration system.

Implement an order lifecycle with transactional state changes.

Simulate database connection exhaustion and resolve it.

10. Redis and Caching

Interview questions

What is Redis?

Why is Redis fast?

What are Redis data structures?

Difference between String, Hash, List, Set and Sorted Set?

What is Redis TTL?

What is cache-aside?

What is write-through caching?

What is write-behind caching?

What is cache invalidation?

What is cache stampede?

What is cache penetration?

What is cache avalanche?

What is Redis persistence?

Difference between RDB and AOF?

What is Redis replication?

What is Redis Sentinel?

What is Redis Cluster?

What is Redis Pub/Sub?

What are Redis Streams?

How do you handle Redis failure?

How do you prevent stale data?

How do you implement distributed locking?

What is the Redlock algorithm?

What happens when Redis memory is full?

How do you implement atomic operations in Redis?

Practical problems

Implement a Redis cache using Go.

Build a cache-aside implementation.

Implement TTL-based caching.

Implement a distributed lock.

Implement a Redis-based rate limiter.

Implement a sliding-window rate limiter.

Implement a leaderboard using Sorted Sets.

Implement a Redis Pub/Sub messaging system.

Implement Redis Streams consumer processing.

Implement cache invalidation after database updates.

Build a Redis-backed order book.

Handle Redis downtime with database fallback.

11. Kafka and Messaging Systems

Interview questions

What is Kafka and why is it used?

What is a Kafka broker?

What is a topic?

What is a partition?

What is an offset?

What is a consumer group?

How does partition assignment work?

What happens when a consumer crashes?

What is consumer rebalancing?

What is offset committing?

Difference between auto commit and manual commit?

What is at-most-once delivery?

What is at-least-once delivery?

What is exactly-once processing?

How do you maintain message ordering?

How do you handle duplicate messages?

What is a dead-letter queue?

What is a retry topic?

What is backpressure in Kafka consumers?

How do you handle poison messages?

What is the difference between Kafka and RabbitMQ?

What is the outbox pattern?

How do you handle Kafka broker failures?

How do you prevent message loss?

How do you handle database failures after consuming Kafka messages?

Practical problems

Create a Kafka producer in Go.

Create a Kafka consumer in Go.

Implement a producer-consumer application.

Implement manual offset commits.

Implement message retry logic.

Implement a dead-letter queue.

Implement idempotent message processing.

Implement ordered message processing.

Implement concurrent Kafka consumers.

Implement graceful shutdown of Kafka consumers.

Implement a Kafka-to-PostgreSQL data pipeline.

Implement a Kafka-to-Redis state updater.

Implement an outbox pattern.

Handle consumer crashes during message processing.

Build a reliable order event processing system.

12. Distributed Systems and System Design

Interview questions

What is a distributed system?

What is CAP theorem?

Explain consistency, availability and partition tolerance.

What is eventual consistency?

What is strong consistency?

What is distributed locking?

What is idempotency?

What is the Saga pattern?

What is the Two-Phase Commit protocol?

What is the transactional outbox pattern?

What is the circuit breaker pattern?

What is bulkhead isolation?

What is exponential backoff?

What is jitter in retry mechanisms?

What is service discovery?

What is load balancing?

What is horizontal scaling?

What is vertical scaling?

What is a distributed cache?

How do you prevent duplicate processing?

How do you handle partial failures?

How do you maintain consistency between Redis and PostgreSQL?

How do you handle network partitions?

How do you design fault-tolerant microservices?

How do you design a high-throughput trading platform?

Practical system design problems

No.	Problem	Level
1	Design URL Shortener	Medium
2	Design Pastebin	Medium
3	Design Rate Limiter	Medium
4	Design Notification Service	Medium
5	Design Distributed ID Generator	Medium
6	Design Job Scheduler	Hard
7	Design Message Queue	Hard
8	Design Distributed Cache	Hard
9	Design Stock Trading Platform	Hard

No.	Problem	Level
10	Design Order Management System	Hard
11	Design Wallet and Transaction System	Hard
12	Design Real-time Leaderboard	Hard
13	Design Inventory Management System	Hard
14	Design High-throughput Event Processing System	Hard

13. Testing, Debugging and Performance

Interview questions

How does Go testing work?

What is a table-driven test?

What is the difference between unit testing and integration testing?

What are benchmarks in Go?

What is go test -race?

What is mocking?

What is dependency injection in testing?

What is test coverage?

What is fuzz testing?

How do you debug a Go application?

What is pprof?

How do you identify CPU bottlenecks?

How do you identify memory leaks?

What is go tool trace?

How do you optimize high-throughput APIs?

How do you investigate high latency?

How do you identify goroutine leaks?

What are flame graphs?

What is the difference between profiling and benchmarking?

How do you investigate production incidents?

Practical problems

Write unit tests for a REST API.

Write table-driven tests for a service.

Mock a PostgreSQL repository.

Mock a Kafka producer.

Write integration tests using PostgreSQL and Redis.

Write concurrent tests for a thread-safe map.

Find a race condition using the race detector.

Benchmark two different implementations.

Profile a CPU-intensive application.

Identify and fix a goroutine leak.

Write fuzz tests for an input parser.

Simulate Kafka and Redis failures.

Perform load testing on a Go REST API.

14. Deployment, Docker, Kubernetes and Production

Interview questions

How do you compile a Go application?

What is a static binary?

How do you build a multi-stage Docker image?

What is the difference between CMD and ENTRYPOINT?

How do you manage environment variables?

What is Kubernetes?

What is a Pod?

What is a Deployment?

What is a Service?

What is ConfigMap?

What is Secret?

What are liveness and readiness probes?

How do you implement graceful shutdown in Kubernetes?

How do you manage rolling deployments?

What happens when a Pod crashes?

How do you manage resource limits?

How do you troubleshoot OOMKilled containers?

How do you handle application logs?

How do you monitor Go applications?

How do you deploy zero-downtime services?

Practical problems

Dockerize a Go REST API.

Create a multi-stage Docker build.

Deploy a Go service to Kubernetes.

Configure readiness and liveness probes.

Implement graceful shutdown with SIGTERM.

Configure CPU and memory limits.

Implement structured JSON logging.

Expose application metrics using Prometheus.

Build a CI/CD pipeline.

Troubleshoot a crashing Go application in Kubernetes.

15. Advanced Go Topics

Interview questions

What are generics in Go?

What are type constraints?

What is the comparable constraint?

What are reflection and the reflect package?

What is unsafe?

What are build tags?

What are Go workspaces?

What are cgo and CGO_ENABLED?

What are atomic values?

What are custom marshalers and unmarshalers?

How does JSON serialization work?

How do you optimize JSON parsing?

What are functional options?

What are custom iterators?

What is the difference between compile-time and runtime behavior?

Practical problems

Implement a generic stack.

Implement a generic queue.

Build a generic repository.

Implement a custom JSON marshaler.

Implement a functional options pattern.

Build a reflection-based struct validator.

Implement a generic concurrent map.

Build a custom configuration loader.

Implement a custom iterator.

Benchmark reflection versus direct field access.

Complete practical coding checklist

This is the consolidated coding list you should actually implement in Go.

Your Go coding tracker

0/64 completed

1. Go fundamentals

* 0 / 7 completed

Reverse a string

Palindrome check

Find second-largest element

Find missing number

Remove duplicates from slice

Merge sorted slices

Rotate array by K positions

2. Data structures

* 0 / 8 completed

Implement stack

Implement queue

Implement linked list

Implement binary search tree

Implement hash set

Implement LRU cache

Implement LFU cache

Implement min heap

3. Concurrency

* 0 / 13 completed

Odd-even printing using goroutines

Producer-consumer

Worker pool

Bounded worker pool

Concurrent counter

Thread-safe bank account

Semaphore

Fan-in / Fan-out

Pipeline

Rate limiter

Concurrent job queue

Graceful shutdown

Goroutine leak detection

4. Backend components

* 0 / 15 completed

CRUD REST API using Gin

JWT middleware

Request validation

Centralized error handling

API rate limiter

Idempotency middleware

PostgreSQL connection pool

Database transaction

Optimistic locking

Redis cache-aside

Redis distributed lock

Kafka producer and consumer

Kafka retry and DLQ

Kafka to PostgreSQL pipeline

Transactional outbox

5. Advanced systems

* 0 / 11 completed

Distributed ID generator

URL shortener

Pastebin

Notification service

Job scheduler

Message queue

Distributed cache

Stock order processor

Order management system

Wallet transaction system

High-throughput event processor

6. Testing and production

* 0 / 10 completed

Table-driven tests

Mock repository

Integration tests

Race-condition debugging

CPU profiling

Memory profiling

Benchmark optimization

Dockerize Go application

Kubernetes deployment

Graceful shutdown with SIGTERM

Reset progress

Recommended preparation priority

For a senior Go backend interview, I would prioritize these areas

Priority

Topics

1

Goroutines, channels, concurrency

2

Slices, maps, pointers and interfaces

3

Mutex, WaitGroup, atomic and race conditions

4

Memory management and garbage collection

5

Context, errors and graceful shutdown

6

REST APIs, Gin and middleware

7

PostgreSQL and Redis

8

Kafka and event-driven architecture

9

System design and distributed systems

10

Testing, profiling and Kubernetes

My recommendation: Don't just memorize 300+ questions. For every topic, understand the internal working, write a small Go program, and then solve a production-oriented scenario.

I can also send you a fresh set of practical Go coding interview problems every day, with test cases and solutions after you attempt them.

Daily Go coding interview practice

Integrated foundations and API engineering

Go Fundamentals and Production Code Structure

Objective Understand Go deeply enough to write maintainable production services.

1. Go language fundamentals

- Go compilation process.
- Source files, packages and modules.
- go.mod and go.sum.
- Variables and constants.
- Static typing and type inference.
- Primitive types.
- Zero values.
- Type conversion.
- Constants and iota.

- Pointers and memory addresses.
- Value semantics versus reference-like behavior.
- Arrays and slices.
- Slice length and capacity.
- Slice backing arrays.
- Slice append behavior.
- Maps and map internals.
- Strings and UTF-8.
- Runes and bytes.
- Structs and struct tags.
- Anonymous structs.
- Embedded structs.
- Methods and receivers.
- Pointer receivers versus value receivers.
- Functions and multiple return values.
- Named return values.
- Variadic functions.
- Closures.
- Higher-order functions.

2. Interfaces and abstraction

- Interface types.
- Implicit interface implementation.
- Interface composition.
- Empty interface and any.
- Type assertions.
- Type switches.
- Interface values internally.
- Dynamic type and dynamic value.
- Typed nil problem.
- Nil interface versus nil pointer.
- Interface segregation.
- Dependency inversion.
- Compile-time interface checks.
- When not to use interfaces.

Example: Define an order repository interface and implement it using in-memory storage and PostgreSQL.

3. Error handling

- error interface.
- Creating errors using errors.New.

- fmt.Errorf.
- Error wrapping with %w.
- errors.Is.
- errors.As.
- Sentinel errors.
- Custom error types.
- Error propagation.
- Error context.
- When to panic.
- Why production business logic should generally return errors instead of panicking.

4. Advanced language features

- Generics.
- Type parameters.
- Type constraints.
- Generic functions.
- Generic data structures.
- comparable.
- defer.
- panic.
- recover.
- Package initialization.
- init().
- Reflection.
- Context package.
- Functional options pattern.

5. Production code organization

- Layered architecture.
- Clean architecture.
- Hexagonal architecture.
- Domain-driven design basics.
- Repository pattern.
- Service layer.
- Handler/controller layer.
- Dependency injection.
- Configuration management.
- Environment variables.
- Structured logging.
- Graceful shutdown.

- Health checks.
- Readiness checks.
- Build an in-memory order management service.

Required operations

- Create order.
- Modify order.
- Cancel order.
- Get order.
- Get all orders.
- Validate order.
- Return structured errors.
- Use interfaces to separate business logic from storage.

Deliverable: A properly structured Go project with unit tests and no global mutable application state.

Go Concurrency and Goroutines

Objective Understand how Go handles concurrency and how to build concurrent services safely.

1. Concurrency fundamentals

- Concurrency versus parallelism.
- Processes versus threads.
- Goroutines.
- Goroutine lifecycle.
- Goroutine creation.
- Goroutine scheduling.
- Cooperative and asynchronous preemption.
- Goroutine stacks.
- Stack growth.
- User-space scheduling.
- OS threads.
- GOMAXPROCS.

2. Go scheduler internals

Understand the G-M-P model

- G: Goroutine.
- M: OS thread.
- P: Processor context required to execute Go code.
- Run queues.
- Global run queue.
- Local run queues.
- Work stealing.

- Syscalls.
- Blocking goroutines.
- Thread handoff.
- Scheduler preemption.
- Network poller.
- Timer management.
- Understand why creating 100,000 goroutines is different from creating 100,000 OS threads.

3. Synchronization primitives

- Mutex.
- RWMutex.
- WaitGroup.
- Once.
- Cond.
- Atomic operations.
- Channels.
- Buffered channels.
- Unbuffered channels.
- Channel direction.
- Closing channels.
- Nil channels.
- Select statement.
- Select with default.
- Channel ownership.
- Producer-consumer patterns.

4. Concurrency problems

- Race conditions.
- Data races.
- Deadlocks.
- Livelocks.
- Starvation.
- Lost updates.
- Double processing.
- Shared mutable state.
- Lock contention.
- Priority inversion.
- Goroutine leaks.

5. Context and cancellation

- context.Background().

- context.TODO().
- context.WithCancel.
- context.WithTimeout.
- context.WithDeadline.
- context.WithValue.
- Cancellation propagation.
- Request-scoped context.
- Parent-child context relationships.
- Context cancellation during shutdown.
- Build a worker pool.
- Build a concurrent order processor.
- Implement a rate-limited API caller.
- Implement graceful worker shutdown.
- Simulate 10,000 concurrent order requests.

Deliverable: A concurrent processing service with bounded goroutines, cancellation and race-free shared state.

Go Memory Management, Garbage Collection and Performance

Objective Understand what happens inside Go when your service executes.

1. Memory fundamentals

- Stack versus heap.
- Stack frames.
- Stack allocation.
- Heap allocation.
- Escape analysis.
- Pointer escape.
- Allocation patterns.
- Memory alignment.
- Object lifetime.
- Memory fragmentation.
- Memory leaks.
- Retained references.

2. Go memory model

- Happens-before relationships.
- Synchronization.
- Memory visibility.
- Atomic operations.
- Data race guarantees.
- Channel synchronization.

- Mutex synchronization.
- Safe publication of data.

3. Garbage collection

- Why garbage collection exists.
- Mark-and-sweep.
- Concurrent garbage collection.
- Tri-color marking.
- Write barriers.
- Root objects.
- Reachability.
- Garbage collection cycles.
- GC pacing.
- GC assists.
- Stop-the-world phases.
- GC CPU overhead.
- Heap growth.
- GOGC.
- GOMEMLIMIT.
- runtime.ReadMemStats.

4. Performance optimization

- CPU profiling.
- Memory profiling.
- Heap profiling.
- Goroutine profiling.
- Block profiling.
- Mutex profiling.
- Execution tracing.
- Benchmarking.
- Allocation profiling.
- pprof.
- go test -bench.
- go test -race.
- go tool trace.
- Build a high-throughput JSON API.
- Benchmark different serialization approaches.
- Compare mutex and channel-based designs.
- Identify memory allocations.
- Find a goroutine leak.

- Fix a data race.
- Profile a CPU-intensive operation.

Deliverable: A benchmark report with CPU, memory, latency and allocation measurements.

Data Structures and Algorithms

Objective Develop problem-solving ability for senior backend interviews.

1. Complexity analysis

- Big O notation.
- Time complexity.
- Space complexity.
- Best, average and worst cases.
- Amortized analysis.
- Recurrence relations.
- Recursion trees.

2. Arrays and strings

- Two pointers.
- Sliding window.
- Prefix sums.
- Difference arrays.
- Kadane's algorithm.
- Binary search.
- Rotated arrays.
- Sorting.
- Merge sort.
- Quick sort.
- Heap sort.
- String manipulation.
- Anagrams.
- Palindromes.

3. Linked lists

- Singly linked lists.
- Doubly linked lists.
- Reversal.
- Cycle detection.
- Fast and slow pointers.
- Merging sorted lists.
- LRU cache implementation.

4. Stack and queue

- Stack.
- Queue.
- Deque.
- Monotonic stack.
- Monotonic queue.
- Expression evaluation.
- Parentheses validation.
- Sliding window maximum.

5. Trees and graphs

- Binary trees.
- Binary search trees.
- Tree traversals.
- BFS.
- DFS.
- Lowest common ancestor.
- Balanced trees.
- Heaps.
- Tries.
- Graph representation.
- Topological sorting.
- Dijkstra.
- Bellman-Ford.
- Union-Find.
- Cycle detection.

6. Dynamic programming

- Memoization.
- Tabulation.
- 1D DP.
- 2D DP.
- Knapsack.
- Longest common subsequence.
- Longest increasing subsequence.
- Coin change.
- Grid problems.

Deliverable: Solve approximately 50-70 carefully selected problems, focusing on patterns rather than memorizing solutions.

Computer Networking, HTTP and API Design

Objective Understand how requests travel from clients to backend services.

1. Networking fundamentals

- OSI model.
- TCP/IP model.
- IP addresses.
- Public and private IPs.
- IPv4 versus IPv6.
- Subnetting.
- CIDR.
- MAC addresses.
- ARP.
- Routing.
- NAT.
- Ports.
- Sockets.
- DNS.
- DHCP.

2. TCP internals

- TCP handshake.
- SYN, SYN-ACK, ACK.
- TCP connection establishment.
- Connection termination.
- FIN and RST.
- Sequence numbers.
- Acknowledgements.
- Retransmissions.
- Sliding window.
- Flow control.
- Congestion control.
- TCP slow start.
- Congestion avoidance.
- Head-of-line blocking.
- Keepalive.
- TIME_WAIT.
- Connection reuse.

3. UDP

- UDP versus TCP.
- Datagram communication.
- Packet loss.
- Ordering.
- Reliability.
- DNS over UDP.
- Real-time communication.
- QUIC and its relationship with UDP.

4. HTTP

- HTTP/1.0.
- HTTP/1.1.
- HTTP/2.
- HTTP/3.
- Request-response lifecycle.
- HTTP methods.
- GET, POST, PUT, PATCH, DELETE.
- Idempotency.
- Safe methods.
- HTTP headers.
- Cookies.
- Sessions.
- Status codes.
- Content negotiation.
- Compression.
- Chunked transfer encoding.
- Keep-alive.
- HTTP caching.
- ETags.
- Conditional requests.
- Range requests.

5. TLS and HTTPS

- Encryption.
- Symmetric encryption.
- Asymmetric encryption.
- Public/private keys.
- Digital signatures.
- Hashing.

- Certificates.
- Certificate authorities.
- Certificate chains.
- TLS handshake.
- TLS 1.2 versus 1.3.
- Certificate validation.
- Mutual TLS.
- Certificate rotation.

6. API design

- REST.
- RPC.
- gRPC.
- GraphQL.
- API versioning.
- Pagination.
- Filtering.
- Sorting.
- Request validation.
- Response structures.
- Error contracts.
- Idempotency keys.
- API authentication.
- API authorization.
- Rate limits.
- Backward compatibility.
- OpenAPI/Swagger.

Practical assignment: Build an HTTP order API with validation, pagination, request IDs, authentication and idempotent order placement.

Infrastructure, caching and event-driven systems

Absolutely. For a Senior Backend Engineer (5+ years), Docker, Redis and Kafka are important because interviewers often test not just definitions but also internal architecture, failure handling, performance and production troubleshooting.

Here is a dedicated preparation list with theory questions + practical coding problems + real-world scenarios.

1. Docker interview questions

Docker Architecture (What Actually Happens Internally)

A. Docker fundamentals

1. What is Docker?
2. Why do we need containers?
3. What is the difference between a container and a virtual machine?
4. What is a Docker image?
5. What is a Docker container?
6. What is Docker Engine?
7. Explain Docker architecture.
8. What is Docker Hub?
9. What is a Docker registry?
10. What is the difference between an image and a container?
11. What are Docker layers?
12. What is a Dockerfile?
13. What is Docker Compose?
14. What is the difference between Docker and Kubernetes?
15. What happens internally when you execute docker run?

B. Dockerfile and image optimization

1. What is the difference between RUN, CMD and ENTRYPOINT?
2. What is the difference between COPY and ADD?
3. What is a multi-stage Docker build?
4. Why should we use Alpine or distroless images?
5. How do you reduce Docker image size?
6. What is Docker layer caching?
7. How does Docker cache work?
8. What is .dockerignore?
9. How do you pass environment variables?
10. How do you manage secrets in Docker?
11. What is the difference between build-time and runtime arguments?

12. What happens if you change an instruction in the middle of a Dockerfile?

13. What is a non-root container?

14. How do you scan Docker images for vulnerabilities?

C. Docker networking and storage

1. What is Docker bridge networking?

2. What is host networking?

3. What is a Docker network?

4. How do containers communicate with each other?

5. What is port mapping?

6. Difference between EXPOSE and -p?

7. What are Docker volumes?

8. Difference between volumes and bind mounts?

9. What happens to container data after container deletion?

10. How do you persist PostgreSQL or Redis data?

11. How does DNS resolution work between containers?

12. What is the difference between bridge and overlay networks?

D. Docker production and troubleshooting

1. What happens when a container exits?

2. How do you restart a failed container?

3. What are Docker health checks?

4. What is the difference between health checks and readiness probes?

5. How do you troubleshoot a container that keeps restarting?

6. How do you investigate high CPU usage?

7. How do you investigate memory usage?

8. What is OOMKilled?

9. How do you handle graceful shutdown?

10. How do you pass SIGTERM to a Go application?

11. What is PID 1 inside a container?

12. What is the difference between docker stop and docker kill?

13. How do you handle container logs?

14. How do you configure CPU and memory limits?

15. How do you implement zero-downtime deployment?

Docker practical problems

Problem Difficulty

 * 1 Dockerize a Go REST API Easy

 * 2 Run PostgreSQL using Docker Easy

 * 3 Run Redis using Docker Easy

 * 4 Create a Dockerfile for Go Medium

 * 5 Create a multi-stage Docker build Medium

 * 6 Create Docker Compose for Go + Redis + PostgreSQL Medium

 * 7 Configure persistent Redis storage Medium

 * 8 Create a custom Docker network Medium

 * 9 Configure container health checks Medium

 * 10 Optimize Docker image size Medium

 * 11 Debug a container restarting continuously Hard

 * 12 Resolve container-to-container connectivity issues Hard

 * 13 Handle graceful shutdown in Docker Hard

 * 14 Debug OOMKilled containers Hard

 * 15 Build and deploy a production Go container Hard

Hands-on Docker assignment

Build this environment

Go backend development environment

Go REST API

Gin · Port 8080

Redis

Cache · Port 6379

PostgreSQL

Database · Port 5432

All services run using Docker Compose with persistent volumes and a shared network.

Requirements

* Create a Go Dockerfile.

* Create docker-compose.yml.

* Add Redis and PostgreSQL.

* Configure environment variables.

* Add health checks.

* Configure persistent volumes.

* Implement graceful shutdown.

* Ensure the Go service waits for dependencies to become healthy.

2. Redis interview questions

Redis Explained: A Complete Beginner's Guide (2026) | Xcademia

xcademia.com

A. Redis fundamentals

1. What is Redis?
2. Why is Redis faster than traditional databases?
3. Is Redis an in-memory database?
4. What data structures does Redis support?
5. Difference between Redis and Memcached?
6. What is Redis key expiration?
7. What is TTL?
8. What happens when a key expires?
9. What happens when Redis runs out of memory?
10. What is the difference between SET, MSET and SETNX?
11. What is the difference between GET and MGET?
12. What is pipelining?
13. What are Redis transactions?
14. What is the difference between MULTI/EXEC and Lua scripts?
15. What is Redis atomicity?

B. Redis internals

1. How does Redis store data internally?
2. Why is Redis single-threaded for command execution?
3. Does Redis support multithreading?
4. What are SDS (Simple Dynamic Strings)?
5. What is a ziplist/listpack?
6. How does Redis implement Hash?
7. How does Redis implement Sorted Sets?
8. What is a skip list?
9. What is a hash table?
10. What is incremental rehashing?
11. How does Redis handle expiration?

12. What are lazy expiration and active expiration?

13. What is copy-on-write?

14. How does Redis persistence work?

15. What is the difference between RDB and AOF?

C. Redis caching

1. What is cache-aside?

2. What is read-through caching?

3. What is write-through caching?

4. What is write-behind caching?

5. What is cache invalidation?

6. What is cache stampede?

7. What is cache penetration?

8. What is cache avalanche?

9. What is stale cache data?

10. How do you handle cache misses?

11. How do you synchronize Redis and PostgreSQL?

12. What happens if the database update succeeds but Redis update fails?

13. How do you prevent stale reads?

14. What is a cache warming strategy?

15. How do you select a TTL?

D. Redis distributed systems

1. What is Redis replication?

2. What is Redis Sentinel?

3. What is Redis Cluster?

4. Difference between Sentinel and Cluster?

5. How does Redis failover work?

6. What is the Redis replication backlog?

7. What is a replication offset?

8. What are hash slots?

9. What is the difference between Pub/Sub and Streams?

10. What is consumer group in Redis Streams?

11. What is a distributed lock?

12. Explain SET key value NX PX.

13. What happens if a distributed lock expires while a process is still working?

14. What is the Redlock algorithm?

15. How do you handle Redis node failure?

Redis practical coding problems

Problem Difficulty

 * 1 Implement Redis GET/SET using Go Easy
 * 2 Implement TTL-based cache Easy
 * 3 Implement cache-aside pattern Medium
 * 4 Implement LRU cache Medium
 * 5 Implement sliding-window rate limiter Medium
 * 6 Implement token-bucket rate limiter Medium
 * 7 Implement distributed lock Medium
 * 8 Implement Redis Pub/Sub Medium
 * 9 Implement Redis Streams consumer Medium
 * 10 Implement leaderboard using Sorted Sets Medium
 * 11 Implement cache invalidation Medium
 * 12 Prevent cache stampede Hard
 * 13 Build a Redis-backed job queue Hard
 * 14 Build a distributed rate limiter Hard
 * 15 Implement Redis failover handling Hard

Redis real-world scenarios

Scenario 1: Redis goes down

Your Go trading API uses Redis to store order state.

* How will the application behave?

* Should you fall back to PostgreSQL?

* How do you avoid overloading PostgreSQL?

* How do you recover Redis state?

* How do you prevent duplicate order processing?

Scenario 2: Cache inconsistency

PostgreSQL contains order status COMPLETED, but Redis contains PENDING.

* Which data source is authoritative?

* How will you reconcile the state?

* Would you use Kafka to update Redis?

* How do you handle out-of-order events?

Scenario 3: High Redis latency

Redis latency increases from 1 ms to 200 ms.

* How will you identify the problem?

* How do you inspect slow commands?

* How do you inspect memory?

* How do you identify hot keys?

* How do you monitor connection pools?

3. Kafka interview questions

All things Kafka - DEV Community

A. Kafka fundamentals

1. What is Kafka?
2. Why is Kafka used?
3. What is a broker?
4. What is a topic?
5. What is a partition?
6. What is an offset?
7. What is a producer?
8. What is a consumer?
9. What is a consumer group?
10. How does Kafka maintain ordering?
11. What is a message key?
12. What is a partition key?
13. What is a replication factor?
14. What is a leader partition?
15. What are follower replicas?
16. What is ISR (In-Sync Replicas)?
17. What is retention policy?
18. What is log compaction?
19. What is consumer lag?
20. What is the difference between Kafka and RabbitMQ?

B. Kafka internals

1. Explain Kafka architecture.
2. How does Kafka store messages?
3. What is a commit log?
4. How does Kafka achieve high throughput?
5. What is zero-copy transfer?
6. What is sequential disk I/O?

7. What is page cache?

8. How does Kafka replication work?

9. What happens when a broker goes down?

10. What is controller in Kafka?

11. What is KRaft?

12. What was ZooKeeper used for?

13. What is leader election?

14. What is a partition reassignment?

15. What is a rebalance?

16. What is cooperative rebalancing?

17. What is the difference between acks=0, acks=1 and acks=all?

18. What is min.insync.replicas?

19. What is idempotent producer?

20. What is producer batching?

C. Kafka consumers and delivery guarantees

1. What is auto offset commit?

2. What is manual offset commit?

3. What happens if a consumer crashes before committing an offset?

4. What happens if a consumer commits an offset before processing?

5. Explain at-most-once delivery.

6. Explain at-least-once delivery.

7. Explain exactly-once semantics.

8. How do you prevent duplicate processing?

9. How do you handle poison messages?

10. What is a Dead Letter Queue?

11. What is a retry topic?

12. How do you preserve message ordering?

13. What happens when consumer processing takes longer than expected?

14. How do you handle consumer lag?

15. What is backpressure?

16. How do you scale Kafka consumers?

17. What happens when there are more consumers than partitions?

18. How do you handle database failures during message processing?

19. What is transactional messaging?

20. What is the outbox pattern?

D. Kafka practical coding problems

Problem Difficulty

* 1 Implement Kafka producer in Go Easy
* 2 Implement Kafka consumer in Go Easy
* 3 Consume messages using consumer groups Medium
* 4 Implement manual offset commits Medium
* 5 Implement retry mechanism Medium
* 6 Implement Dead Letter Queue Medium
* 7 Implement idempotent consumer Medium
* 8 Implement ordered processing Medium
* 9 Implement concurrent message processing Medium
* 10 Implement graceful consumer shutdown Medium
* 11 Implement Kafka to PostgreSQL pipeline Hard
* 12 Implement Kafka to Redis state updater Hard
* 13 Implement transactional outbox Hard
* 14 Implement event replay mechanism Hard
* 15 Implement reliable order event processing Hard

Kafka real-world scenarios

Scenario 1: Duplicate order events

Kafka delivers the same order event twice.

* How do you prevent duplicate database updates?
* Would you use an idempotency key?
* Would you maintain a processed-event table?
* How do you handle concurrent duplicate consumers?

Scenario 2: Consumer crashes

A consumer processes a message, updates PostgreSQL and crashes before committing the offset.

* What happens when the consumer restarts?
* Will the event be processed again?
* How do you ensure the database update is idempotent?

Scenario 3: Consumer lag increases

Your consumer lag increases from 1,000 to 1 million messages.

* How do you investigate?

* Should you increase consumer instances?

* What happens if the number of consumers exceeds partitions?

* How do you optimize processing throughput?

* How do you avoid overwhelming PostgreSQL?

Scenario 4: Maintaining order lifecycle

An order generates events

PLACED → MODIFIED → CANCELLED

* How do you ensure events are processed in order?

* What partition key would you choose?

* How do you handle delayed events?

* How do you handle a missing event?

4. Combined Docker + Redis + Kafka practical projects

These are particularly valuable because senior interviews often test how multiple technologies work together.

Project 1: Event-driven order processing system

High priority

Agents365-ai/drawio-skill - Hermes Agent Skills & Skill Registries | Hermes Atlas

hermesatlas.com

Architecture

Go Order API → Kafka → Order Consumer → PostgreSQL → Redis State Updater

Requirements

* Create order API in Go.

* Publish order events to Kafka.

* Consume events using worker pools.

* Persist orders in PostgreSQL.

* Update Redis order state.

* Handle duplicate events.

* Implement retry and DLQ.

* Dockerize all services.

* Add graceful shutdown and health checks.

Project 2: Distributed rate limiter

Rate Limiter System Design (Token Bucket Explained) · InterviewCrafted

interviewcrafted.com

Architecture

Multiple Go API instances → Redis shared rate-limit state

Requirements

* Implement token bucket.

* Support concurrent requests.

* Use Redis atomic operations or Lua.

* Support different limits per client.

* Handle Redis failure.

* Run multiple API instances with Docker Compose.

Project 3: Reliable background job processing

How Slack Built a Distributed Cron Execution System for Scale

blog.bytebytego.com

Architecture

Go API → Kafka → Worker Pool → PostgreSQL

Requirements

* Submit jobs.

* Process jobs asynchronously.

* Implement retry and DLQ.

* Prevent duplicate execution.

* Handle worker crashes.

* Monitor job status.

* Scale workers horizontally.

5. Production troubleshooting questions

These are excellent senior-level interview questions.

Problem What interviewer expects

Docker container restarts continuously Logs, exit codes, health checks, startup dependencies

Docker container gets OOMKilled Memory limits, heap profiling, GC, resource analysis

Redis latency increases Slow log, hot keys, memory, network, connection pool

Redis data disappears TTL, eviction, persistence, failover

Kafka consumer lag increases Partition count, consumer throughput, DB bottlenecks

Kafka messages duplicated Idempotency, offset commits, transaction boundaries

Kafka messages lost Producer acknowledgments, ISR, offset handling

PostgreSQL connection exhaustion Pool configuration, leaks, query duration

Go service has high latency pprof, blocking, GC, dependency latency

Service crashes during deployment SIGTERM, graceful shutdown, in-flight requests

6. Your practical preparation roadmap

I recommend this sequence rather than studying all three technologies independently.

4-week preparation plan

1. Week 1 - Docker

* Docker architecture and internals
* Dockerfile and multi-stage builds
* Volumes and networking
* Docker Compose
* Container debugging
* Dockerize Go + PostgreSQL + Redis

2.

3. Week 2 - Redis

* Data structures and internals
* Persistence and replication
* Caching strategies
* Distributed locks
* Rate limiter
* Redis failure and recovery

4.

5. Week 3 - Kafka

* Broker, topic and partitions
* Consumer groups and offsets
* Delivery guarantees
* Retry and DLQ
* Idempotency
* Kafka + Go + PostgreSQL integration

6.

7. Week 4 - Integration and system design

* Build the event-driven order platform
* Test duplicate events
* Kill Redis and Kafka during processing
* Test consumer crashes
* Perform load testing
* Analyze latency and throughput

8.

7. Master practical checklist

Docker + Redis + Kafka tracker

0/36

Docker

Write a Go Dockerfile

Create a multi-stage build

Build Docker Compose for Go, Redis and PostgreSQL

Configure volumes and networking

Implement health checks

Debug container restarts

Handle SIGTERM and graceful shutdown

Debug OOMKilled containers

Redis

Implement cache-aside

Implement LRU cache

Implement token bucket rate limiter

Implement sliding window rate limiter

Implement distributed lock

Implement Pub/Sub

Implement Streams consumer groups

Prevent cache stampede

Handle Redis failover

Build Redis-backed order state

Kafka

Build Go producer

Build Go consumer

Implement manual offset commit

Implement retry and DLQ

Implement idempotent consumer

Maintain order using partition keys

Implement concurrent workers

Handle consumer rebalancing

Implement transactional outbox

Build Kafka to Redis/PostgreSQL pipeline

Integrated projects

Event-driven order processing system

Distributed rate limiter

Reliable background job processor

Test duplicate Kafka messages

Simulate Redis downtime

Simulate Kafka broker failure

Test PostgreSQL connection exhaustion

Load test multiple Go API instances

Reset checklist

My recommendation: Given your Go trading backend experience, spend extra time on Kafka consumer reliability, Redis state consistency, Docker/Kubernetes production debugging, and graceful shutdown. These are the areas where you can demonstrate genuine senior-level engineering depth.

You can use your existing order-processing experience to explain these scenarios with real architectural trade-offs rather than giving textbook answers.

Integrated infrastructure and messaging engineering

Reverse Proxy, Load Balancer, CDN and Edge Infrastructure

Objective Understand how large-scale applications distribute and serve traffic.

1. Forward and reverse proxies

- Proxy fundamentals.
- Forward proxy.
- Reverse proxy.
- Proxy authentication.
- Request routing.
- Header forwarding.
- X-Forwarded-For.
- X-Forwarded-Proto.
- Proxy buffering.
- Connection pooling.

2. Load balancing

- Layer 4 load balancing.
- Layer 7 load balancing.
- Round robin.
- Weighted round robin.
- Least connections.
- Least response time.
- Consistent hashing.
- IP hash.
- Sticky sessions.
- Health checks.
- Active health checks.
- Passive health checks.
- Failover.
- Connection draining.
- Load balancer high availability.

3. CDN - Content Delivery Network

- Why CDNs exist.
- Origin servers.

- Edge servers.
- Points of presence.
- Edge caching.
- Cache hierarchy.
- Cache hit and cache miss.
- Cache-control headers.
- TTL.
- Cache invalidation.
- Purging.
- Stale-while-revalidate.
- Stale-if-error.
- Origin shield.
- Request collapsing.
- Cache key.
- Query parameter normalization.
- Geographic routing.
- Anycast.
- Dynamic versus static content.
- CDN security.
- CDN origin protection.

4. DNS and traffic routing

- DNS resolution.
- Recursive resolvers.
- Authoritative DNS.
- A, AAAA, CNAME, MX, TXT records.
- DNS TTL.
- DNS caching.
- GeoDNS.
- Latency-based routing.
- Weighted DNS routing.
- Failover routing.
- Anycast routing.

5. WAF and edge security

- Web Application Firewall.
- DDoS protection.
- Bot protection.
- IP reputation.
- Geo-blocking.

- Request filtering.
- SQL injection protection.
- Cross-site scripting protection.
- Origin IP protection.
- TLS termination.

6. Infrastructure examples

Understand the roles of

- NGINX.
- HAProxy.
- Envoy.
- AWS Application Load Balancer.
- AWS Network Load Balancer.
- CloudFront.
- Cloudflare.
- Fastly.

Practical assignment: Design the traffic flow

- Client → DNS → CDN → WAF → Load Balancer → Reverse Proxy → Backend Service.
- Explain what happens during cache hit, cache miss, origin failure and regional failure.

Resilience Engineering and Fault Tolerance

Objective Design services that continue working when dependencies fail.

1. Timeout

- Connection timeout.
- Read timeout.
- Write timeout.
- Request timeout.
- Database timeout.
- Context deadline.
- End-to-end deadline.
- Timeout propagation.
- Timeout budgeting.
- Why infinite timeouts are dangerous.

2. Retry pattern

- Retryable errors.
- Non-retryable errors.
- Fixed interval retries.
- Exponential backoff.

- Exponential backoff with jitter.
- Maximum retry count.
- Retry budgets.
- Retry amplification.
- Duplicate execution.
- Idempotent retries.
- Retry storms.

3. Circuit breaker

Understand the three states

- Closed.
- Open.
- Half-open.
- Failure threshold.
- Failure percentage.
- Slow-call threshold.
- Open duration.
- Recovery probing.
- Reset behavior.
- Circuit breaker metrics.
- Dependency isolation.

4. Other resilience patterns

- Bulkhead.
- Rate limiting.
- Throttling.
- Backpressure.
- Load shedding.
- Graceful degradation.
- Fail-fast.
- Fallback.
- Hedged requests.
- Request deduplication.
- Bounded queues.
- Queue overflow.
- Dead-letter queues.

5. Distributed failure scenarios

- Partial failures.
- Network partitions.

- Slow dependencies.
- Cascading failures.
- Retry storms.
- Thundering herd.
- Split brain.
- Resource exhaustion.
- Dependency saturation.

Create a Go service that calls three dependencies

- Risk service.
- Account service.
- OMS service.
- Implement timeout, retries, circuit breakers, bulkheads and graceful degradation.

Deliverable: Demonstrate how the system behaves when each dependency becomes slow, unavailable or returns errors.

Docker Internals

Objective Understand containerization beyond writing Dockerfiles.

1. Docker fundamentals

- Virtual machines versus containers.
- Docker architecture.
- Docker Engine.
- Docker daemon.
- Docker CLI.
- Docker images.
- Containers.
- Registries.
- Image layers.
- Union filesystem.
- Copy-on-write.
- Image caching.

2. Linux internals behind containers

- Namespaces.
- PID namespace.
- Network namespace.
- Mount namespace.
- IPC namespace.
- UTS namespace.
- User namespace.
- Cgroups.

- CPU limits.
- Memory limits.
- I/O limits.
- Linux capabilities.
- Seccomp.
- AppArmor.
- Container isolation.

3. Docker networking

- Bridge networking.
- Host networking.
- None networking.
- Overlay networking.
- Port mapping.
- DNS resolution.
- Container-to-container communication.
- Network isolation.

4. Docker storage

- Container writable layer.
- Volumes.
- Bind mounts.
- tmpfs.
- Persistent storage.
- Volume lifecycle.

5. Production Docker

- Multi-stage builds.
- Minimal images.
- Distroless images.
- Non-root users.
- Health checks.
- Environment configuration.
- Secret management.
- Image scanning.
- Image tagging.
- Build caching.
- Resource limits.
- Logging drivers.

Practical assignment: Containerize your order platform, PostgreSQL, Redis and Kafka using Docker Compose.

Kubernetes Internals

Objective Understand how production containerized systems are deployed and managed.

1. Kubernetes architecture

- Control plane.
- Worker nodes.
- API server.
- Scheduler.
- Controller manager.
- etcd.
- Kubelet.
- Kube-proxy.
- Container runtime.
- Reconciliation loops.
- Desired state.
- Actual state.

2. Kubernetes objects

- Pod.
- ReplicaSet.
- Deployment.
- StatefulSet.
- DaemonSet.
- Job.
- CronJob.
- Service.
- Ingress.
- Gateway API.
- ConfigMap.
- Secret.
- Namespace.
- PersistentVolume.
- PersistentVolumeClaim.
- StorageClass.

3. Networking

- Pod networking.
- Service networking.
- ClusterIP.
- NodePort.

- LoadBalancer.
- Headless services.
- DNS.
- CNI.
- Network policies.
- Ingress controllers.
- Service mesh basics.

4. Scaling and scheduling

- Requests and limits.
- HPA.
- VPA.
- Cluster autoscaling.
- Node affinity.
- Pod affinity.
- Taints and tolerations.
- Topology spread constraints.
- Resource quotas.
- Pod disruption budgets.

5. Reliability and operations

- Liveness probes.
- Readiness probes.
- Startup probes.
- Rolling deployments.
- Blue-green deployment.
- Canary deployment.
- Rollbacks.
- Pod eviction.
- CrashLoopBackOff.
- OOMKilled.
- Pending pods.
- Node failures.
- Graceful termination.
- PreStop hooks.

Practical assignment: Deploy the order service on Kubernetes with three replicas, health checks, resource limits, autoscaling and rolling updates.

Redis Fundamentals and Internals

Objective Understand Redis as an in-memory data platform.

1. Redis architecture

- Redis architecture.
- Event loop.
- Single-threaded command execution.
- I/O threading.
- Network I/O.
- Command execution.
- Pipelining.
- RESP protocol.
- Memory allocation.
- Key expiration.
- Lazy deletion.
- Active expiration.

2. Redis data structures

- Strings.
- Lists.
- Sets.
- Sorted sets.
- Hashes.
- Bitmaps.
- HyperLogLog.
- Streams.
- Geospatial indexes.

3. Redis internals

- SDS.
- Hash tables.
- Listpack.
- Skip lists.
- Intsets.
- Compact encodings.
- Memory overhead.
- Key eviction.
- LRU.
- LFU.
- TTL.
- maxmemory.
- Eviction policies.

4. Redis persistence

- RDB snapshots.
- AOF.
- AOF rewrite.
- Hybrid persistence.
- fsync policies.
- Recovery.
- Persistence trade-offs.
- Durability limitations.

5. Redis transactions and scripting

- MULTI.
- EXEC.
- DISCARD.
- WATCH.
- Optimistic locking.
- Atomic commands.
- Lua scripting.
- Redis Functions.
- Atomic read-modify-write operations.

6. Distributed locking

- SET NX.
- Expiration.
- Lock ownership.
- Random lock tokens.
- Safe release.
- Lock expiration.
- Redlock debate.
- Fencing tokens.
- Distributed lock limitations.

Practical assignment: Implement Redis-backed orderbook, client order state and idempotency storage.

Redis High Availability, Cluster and Caching Strategies

1. Redis replication

- Primary-replica replication.
- Asynchronous replication.
- Replication backlog.
- Replication offset.
- Full synchronization.

- Partial synchronization.
- Replication lag.
- Replica promotion.
- Data loss risks.

2. Redis Sentinel

- Sentinel architecture.
- Monitoring.
- Failure detection.
- Quorum.
- Leader election.
- Failover.
- Split-brain risks.
- Client discovery.

3. Redis Cluster

- Hash slots.
- 16,384 hash slots.
- Key distribution.
- Hash tags.
- Cluster nodes.
- Primary and replica.
- Resharding.
- MOVED.
- ASK.
- Cluster failover.
- Multi-key limitations.

4. Caching strategies

- Cache-aside.
- Read-through.
- Write-through.
- Write-behind.
- Write-around.
- Refresh-ahead.
- Local cache.
- Distributed cache.
- Multi-level cache.

5. Cache problems

- Cache stampede.

- Cache avalanche.
- Cache penetration.
- Hot keys.
- Cold starts.
- Cache invalidation.
- Stale reads.
- Negative caching.
- TTL jitter.
- Dogpile effect.

6. Redis Streams

- Stream entries.
- Consumer groups.
- Consumer names.
- Pending entries.
- Acknowledgement.
- XACK.
- XREADGROUP.
- XAUTOCLAIM.
- Consumer failure recovery.
- Stream trimming.

Design a distributed Redis state layer for

- Orderbook.
- Positions.
- Holdings.
- Trades.
- Funds.
- Client-level order state.
- Define how state is rebuilt after Redis restart and how stale data is handled.

Kafka Architecture and Internals

Objective Understand Kafka beyond producing and consuming messages.

1. Kafka fundamentals

- Event streaming.
- Broker.
- Cluster.
- Topic.
- Partition.
- Replica.

- Leader.
- Follower.
- Offset.
- Log.
- Segment.
- Retention.
- Compaction.

2. Kafka architecture

- Distributed log.
- Partition leadership.
- Replication.
- ISR.
- High watermark.
- Leader epoch.
- Controller.
- Metadata.
- KRaft.
- ZooKeeper legacy architecture.
- Broker failure.
- Leader election.
- Replica synchronization.

3. Producer internals

- Producer buffer.
- Batching.
- Compression.
- Linger.
- Acknowledgements.
- acks=0.
- acks=1.
- acks=all.
- Idempotent producer.
- Retries.
- Sequence numbers.
- Transactions.
- Partitioners.
- Ordering guarantees.

4. Consumer internals

- Consumer groups.

- Group coordinator.
- Partition assignment.
- Rebalancing.
- Heartbeats.
- Session timeout.
- Poll interval.
- Offset management.
- Auto commit.
- Manual commit.
- Rebalance listeners.
- Static membership.

5. Delivery semantics

- At-most-once.
- At-least-once.
- Exactly-once processing.
- Duplicate events.
- Idempotent consumers.
- Transactional producer.
- Kafka transactions.
- External database consistency limitations.

Build a Kafka-based order event system

- Order API → Kafka → Order Processor → PostgreSQL → Redis State Updater
- Implement message keys, retries, manual offset commits and duplicate handling.

Advanced Kafka and Reliable Event Processing

1. Kafka reliability

- Replication factor.
- min.insync.replicas.
- Unclean leader election.
- Producer acknowledgements.
- Consumer lag.
- Under-replicated partitions.
- Offline partitions.
- ISR shrink and expansion.
- Broker recovery.
- Data loss scenarios.

2. Ordering

- Ordering within a partition.

- Ordering across partitions.
- Key-based partitioning.
- Partition expansion effects.
- Per-client ordering.
- Per-order ordering.
- Event sequence numbers.

3. Reliable messaging patterns

- Transactional outbox.
- Transactional inbox.
- CDC.
- Debezium.
- Retry topics.
- Dead-letter topics.
- Delayed retries.
- Poison messages.
- Idempotent consumers.
- Deduplication tables.
- Event replay.
- Reprocessing.

4. Kafka performance

- Throughput.
- Latency.
- Batching.
- Compression.
- Producer buffer memory.
- Fetch size.
- Consumer concurrency.
- Partition count.
- Disk I/O.
- Network throughput.
- Page cache.
- Consumer lag management.

5. Schema management

- JSON schema.
- Avro.
- Protobuf.
- Schema Registry.
- Backward compatibility.

- Forward compatibility.
- Full compatibility.
- Schema evolution.
- Event versioning.

6. Event-driven architecture

- Event notification.
- Event-carried state transfer.
- Event sourcing.
- CQRS.
- Saga.
- Choreography.
- Orchestration.
- Eventual consistency.

Practical assignment: Build rejected-order recovery

- Receive rejected order.
- Publish event to Kafka.
- Persist event in PostgreSQL.
- Update Redis order state.
- Recover from consumer failure.
- Reprocess events without duplicating state.

Database systems and data engineering

Database Interview Preparation - SQL, NoSQL, Document, Cassandra & Time-Series Databases

For a Senior Backend Engineer (5+ years), database interviews go beyond SQL queries. You should understand storage engines, indexing, query optimization, transactions, isolation, replication, partitioning, sharding, consistency and database internals.

I'll cover both relational and non-relational databases, including practical SQL problems, Go implementation exercises and production scenarios.

1. Database fundamentals

Interview questions

What is a database?

What is the difference between DBMS and RDBMS?

What is SQL vs NoSQL?

What are OLTP and OLAP?

What is a schema?

What are tables, rows and columns?

What are primary and foreign keys?

What are candidate keys and composite keys?

What is a unique constraint?

What is normalization?

Explain 1NF, 2NF, 3NF and BCNF.

What is denormalization?

What are database indexes?

What is a clustered index?

What is a non-clustered index?

What is a composite index?

What is a covering index?

What is a partial index?

What is a unique index?

What is a database constraint?

What is a view?

What is a materialized view?

What is a stored procedure?

What is a trigger?

What is a sequence?

What is a database transaction?

Explain ACID properties.

What is database replication?

What is database partitioning?

What is database sharding?

2. MySQL interview questions

MySQL :: MySQL 9.7 Reference Manual :: 17.4 InnoDB Architecture

dev.mysql.com

A. MySQL fundamentals

What is MySQL?

What is the difference between MySQL and PostgreSQL?

What are MySQL storage engines?

What is InnoDB?

Difference between InnoDB and MyISAM?

Why is InnoDB the default storage engine?

What is the MySQL architecture?

What is the MySQL query execution lifecycle?

What is the MySQL buffer pool?

What is the InnoDB tablespace?

What is the redo log?

What is the undo log?

What is the binary log?

Difference between redo log and undo log?

Difference between redo log and binary log?

What is the doublewrite buffer?

What is the change buffer?

What is the adaptive hash index?

What is MVCC?

How does MySQL manage concurrent transactions?

B. Indexing and query optimization

How does a B+ Tree index work?

Why does MySQL use B+ Trees?

What is a clustered index in InnoDB?

Can a table have multiple clustered indexes?

What is a secondary index?

How does a secondary index reference the primary key?

What is a composite index?

Explain the leftmost prefix rule.

What is index cardinality?

What is index selectivity?

When will MySQL not use an index?

What is a full table scan?

What is index condition pushdown?

What is a covering index?

What is index merge?

What is the difference between EXPLAIN and EXPLAIN ANALYZE?

What is a filesort?

How do you optimize a slow query?

Why can too many indexes reduce performance?

How do you identify unused indexes?

C. Transactions and locking

Explain ACID in MySQL.

What are transaction isolation levels?

Explain READ UNCOMMITTED.

Explain READ COMMITTED.

Explain REPEATABLE READ.

Explain SERIALIZABLE.

What is a dirty read?

What is a non-repeatable read?

What is a phantom read?

What is MVCC?

What is a record lock?

What is a gap lock?

What is a next-key lock?

What is an intention lock?

What is a deadlock?

How does InnoDB detect deadlocks?

How do you prevent deadlocks?

What is optimistic locking?

What is pessimistic locking?

What is the difference between SELECT FOR UPDATE and SELECT FOR SHARE?

What is autocommit?

What happens if a transaction crashes halfway?

How do savepoints work?

How do you handle long-running transactions?

D. Replication and scaling

What is MySQL replication?

What is source-replica replication?

What is asynchronous replication?

What is semi-synchronous replication?

What is synchronous replication?

What is GTID?

What is binary log replication?

What is replication lag?

What happens if the primary database fails?

How does failover work?

What is read/write splitting?

What is horizontal partitioning?

What is vertical partitioning?

What is sharding?

What is consistent hashing?

How do you scale MySQL for millions of records?

How do you handle cross-shard transactions?

How do you maintain consistency across replicas?

MySQL practical SQL problems

No.	Problem	Level
1	Find the second-highest salary	Easy

No.	Problem	Level
2	Find duplicate records	Easy
3	Find employees earning above department average	Medium
4	Find the Nth highest salary	Medium
5	Find the top 3 salaries per department	Medium
6	Find consecutive login days	Medium
7	Find customers who never placed orders	Medium
8	Calculate monthly revenue	Medium
9	Find running account balance	Medium
10	Find gaps in sequential IDs	Medium
11	Find overlapping booking intervals	Hard
12	Calculate moving averages	Hard
13	Find the latest order per customer	Medium
14	Rank products by sales using window functions	Medium
15	Detect duplicate financial transactions	Hard

MySQL practical engineering problems

Design an order table with suitable indexes.

Optimize a query scanning 10 million records.

Fix a query that causes a full table scan.

Implement pagination on a large table.

Implement cursor-based pagination.

Resolve database deadlocks.

Implement optimistic locking with a version column.

Implement transactional money transfers.

Implement a database connection pool in Go.

Implement read/write splitting.

Simulate replication lag.

Design a database migration strategy without downtime.

3. PostgreSQL interview questions

Since PostgreSQL is part of your backend stack, prepare its internals alongside MySQL.

PostgreSQL Internals: Inside the Storage Engine - DEV Community

Interview questions

What is PostgreSQL architecture?

What is MVCC in PostgreSQL?

How does PostgreSQL store rows?

What is a heap table?

What is a tuple?

What is TOAST?

What is WAL (Write-Ahead Logging)?

How does PostgreSQL ensure durability?

What are shared buffers?

What is work_mem?

What is maintenance_work_mem?

What is VACUUM?

Difference between VACUUM and VACUUM FULL?

What is autovacuum?

What is table bloat?

What is HOT update?

What is an MVCC dead tuple?

What is transaction ID wraparound?

What is a checkpoint?

What is a replication slot?

Indexing

What is a B-Tree index?

What is a Hash index?

What is a GIN index?

What is a GiST index?

What is a BRIN index?

What is a partial index?

What is an expression index?

What is a composite index?

When should you use a BRIN index?

How do you optimize JSONB queries?

Transactions and replication

Explain PostgreSQL isolation levels.

What is Serializable Snapshot Isolation?

What is a serialization failure?

How do you handle deadlocks?

What is synchronous replication?

What is asynchronous streaming replication?

What is logical replication?

What is physical replication?

What is replication lag?

What is Point-in-Time Recovery (PITR)?

Practical problems

Optimize slow PostgreSQL queries using EXPLAIN ANALYZE.

Implement JSONB filtering.

Implement concurrent order updates.

Implement transactional event publishing.

Identify table bloat.

Resolve autovacuum performance issues.

Design indexes for a high-volume order table.

Implement keyset pagination.

Investigate replication lag.

Implement database backup and recovery.

4. NoSQL Database Interview Questions

SQL vs NoSQL Databases: Key Differences and Use Cases | Koder.ai

koder.ai

Interview questions

What is NoSQL?

Why do we need NoSQL databases?

What are the four major types of NoSQL databases?

Explain key-value databases.

Explain document databases.

Explain wide-column databases.

Explain graph databases.

What are the advantages of NoSQL?

What are the disadvantages of NoSQL?

SQL vs NoSQL: when would you choose each?

What is the CAP theorem?

What is the PACELC theorem?

What is eventual consistency?

What is strong consistency?

What is eventual consistency vs causal consistency?

What is quorum-based replication?

What is read repair?

What is hinted handoff?

What is anti-entropy repair?

What is consistent hashing?

What is horizontal scaling?

How does NoSQL handle partitioning?

How do you handle transactions in NoSQL?

What is denormalization in NoSQL?

How do you choose a NoSQL database for an application?

Practical problems

Design a key-value store.

Design a document-based user profile system.

Design a distributed cache.

Design a shopping cart using NoSQL.

Design a session management system.

Design a distributed leaderboard.

Design a high-throughput event storage system.

Design a scalable notification system.

Design a globally distributed user activity store.

Design a database that supports millions of writes per second (discuss realistic hardware and architecture constraints).

5. MongoDB - Document Database

How MongoDB Works: NoSQL Database with JSON Documents | Omer Karacam posted on the topic | LinkedIn

A. MongoDB fundamentals

What is MongoDB?

What is a document database?

What is BSON?

What is the difference between JSON and BSON?

What is a collection?

What is a document?

What is ObjectId?

What is the difference between SQL tables and MongoDB collections?

What is schema flexibility?

Does MongoDB support schemas?

What is embedding?

What is referencing?

When should you embed documents?

When should you use references?

What is the 16 MB document limit?

What is the difference between find() and aggregate()?

What is the aggregation pipeline?

What is $match?

What is $group?

What is $lookup?

B. MongoDB indexing

What is an index in MongoDB?

What is a single-field index?

What is a compound index?

What is a multikey index?

What is a text index?

What is a geospatial index?

What is a TTL index?

What is a unique index?

What is a partial index?

What is index selectivity?

How do you inspect query execution plans?

What is explain()?

What is a covered query?

How do indexes affect write performance?

How do you optimize a slow MongoDB query?

C. MongoDB internals and distributed architecture

What is WiredTiger?

How does MongoDB store data?

What is a replica set?

What is a primary node?

What is a secondary node?

How does primary election work?

What is an arbiter?

What is write concern?

What is read concern?

What is read preference?

What is journaling?

What is replication lag?

What is sharding?

What is a shard key?

What is a config server?

What is mongos?

What is a chunk in MongoDB sharding?

What is the balancer?

What are multi-document transactions?

What is change streams?

MongoDB practical problems

No.	Problem	Level
1	Insert and retrieve documents	Easy
2	Filter users by multiple fields	Easy
3	Find duplicate documents	Medium
4	Group sales by month using aggregation	Medium
5	Join collections using $lookup	Medium
6	Implement pagination	Medium
7	Implement TTL-based session expiration	Medium
8	Design an embedded order document	Medium
9	Optimize a slow aggregation pipeline	Hard
10	Design a sharded order collection	Hard
11	Handle replica set failover	Hard
12	Implement change stream processing in Go	Hard
13	Implement transactions using Go	Hard
14	Design a high-volume audit log store	Hard

6. Cassandra - Wide-Column Database

Wide-Column Stores: Cassandra's Ring Architecture

Cassandra is especially important for interviews involving high write throughput, distributed databases and geographically distributed applications.

A. Cassandra fundamentals

What is Apache Cassandra?

Why is Cassandra used?

What is a wide-column database?

What is a keyspace?

What is a table in Cassandra?

What is a partition key?

What is a clustering key?

What is a composite primary key?

What is the difference between partition key and clustering key?

What is a token?

What is a token ring?

What is consistent hashing?

What is a coordinator node?

What is a seed node?

What is a virtual node (vnode)?

What is replication factor?

What is a replication strategy?

Difference between SimpleStrategy and NetworkTopologyStrategy?

What is a gossip protocol?

What is the snitch in Cassandra?

B. Cassandra internals

Explain Cassandra architecture.

How does Cassandra write data?

What is a commit log?

What is a Memtable?

What is an SSTable?

What is a Bloom filter?

What is a partition index?

What is compaction?

Explain Size-Tiered Compaction Strategy.

Explain Leveled Compaction Strategy.

Explain Time-Window Compaction Strategy.

What is tombstone?

Why do tombstones affect performance?

What is read repair?

What is hinted handoff?

What is anti-entropy repair?

What is Merkle Tree?

What is a lightweight transaction?

What is Paxos in Cassandra?

What is the difference between Cassandra and traditional relational databases?

C. Cassandra consistency and replication

What is tunable consistency?

Explain ONE, TWO, THREE and QUORUM.

What is LOCAL_QUORUM?

What is ALL consistency?

What is the difference between read consistency and write consistency?

Explain the quorum formula.

What is eventual consistency?

What happens when a node goes down?

How does Cassandra handle node failure?

What is hinted handoff?

What is the difference between strong and eventual consistency?

What is multi-datacenter replication?

What is the CAP theorem in Cassandra?

How do you recover a failed node?

How do you handle inconsistent replicas?

A particularly important interview question

For replication factor $N=3$, what consistency level should be used for reads and writes to achieve quorum intersection?

$$ R+W>N $$

For example, with $R=2$, $W=2$, and $N=3$, the read and write quorums intersect. This is useful for consistency reasoning, although it does not by itself guarantee full linearizability.

D. Cassandra data modeling

Why is data modeling query-driven in Cassandra?

Why should you avoid joins?

Why should you avoid unrestricted filtering?

What is denormalization?

What is a materialized view?

What is a secondary index?

What is a static column?

What is a collection type?

What are frozen collections?

How do you prevent hot partitions?

How do you design a time-series table?

How do you handle large partitions?

How do you select a partition key?

How do you handle data expiration?

What is TTL in Cassandra?

Cassandra practical problems

Design a user activity table.

Design an order history table.

Design a time-series stock price table.

Design a transaction history table.

Design a high-throughput logging system.

Implement Cassandra CRUD operations using Go.

Implement a batch insert operation.

Handle failed writes and retries.

Handle node failure.

Investigate hot partitions.

Investigate tombstone-related latency.

Optimize a large-partition query.

Design a multi-datacenter replication strategy.

Implement a time-bucketed data model.

Compare Cassandra write performance under different consistency levels.

7. Time-Series Databases

Time-Series Databases at Scale: Choosing the Right Store Before Cardinality Breaks You | HackerNoon

Time-series databases are particularly relevant to trading platforms, financial applications, monitoring systems, IoT and real-time analytics.

A. Time-series fundamentals

What is a time-series database?

How is a time-series database different from a relational database?

What is a timestamp?

What is a measurement?

What are tags and fields?

What is a time bucket?

What is a retention policy?

What is downsampling?

What is data aggregation?

What is a high-cardinality problem?

What is a write-heavy workload?

What is a time-series index?

What is compression?

What is partitioning by time?

What is late-arriving data?

How do you handle out-of-order events?

What is a continuous aggregate?

What is a rollup?

How do you design time-series storage?

What is the difference between event time and processing time?

B. InfluxDB

What is InfluxDB?

What is a bucket?

What is a measurement?

What are tags and fields?

What is Flux?

What is the InfluxDB line protocol?

How does InfluxDB store time-series data?

What is retention management?

How does InfluxDB handle downsampling?

How do you optimize high-cardinality measurements?

C. TimescaleDB

What is TimescaleDB?

What is a hypertable?

What is a chunk?

How does TimescaleDB partition data?

What are continuous aggregates?

What is compression in TimescaleDB?

What is chunk skipping?

How does TimescaleDB differ from PostgreSQL?

What is a retention policy?

How do you optimize time-range queries?

D. Other time-series systems

Prepare these at a conceptual level

Prometheus

OpenTSDB

VictoriaMetrics

QuestDB

ClickHouse

Interview questions

How is Prometheus different from InfluxDB?

How is ClickHouse different from Cassandra?

Why would you choose TimescaleDB over MySQL?

Why would you choose QuestDB for financial tick data?

How do column-oriented databases improve analytical workloads?

What are high-cardinality labels?

What is the difference between metrics and events?

How do you store historical market prices efficiently?

Time-series practical problems

Store stock prices with timestamps.

Calculate 1-minute OHLC candles from tick data.

Calculate 5-minute and 15-minute candles.

Calculate moving averages.

Implement time-bucket aggregation.

Implement data retention and expiration.

Implement downsampling.

Handle out-of-order tick events.

Handle duplicate market data events.

Design a historical LTP storage system.

Design a high-throughput IoT metrics system.

Design a real-time trading analytics database.

Implement time-range pagination.

Optimize a query scanning one year of tick data.

Compare PostgreSQL, TimescaleDB and Cassandra for market data.

8. Other NoSQL Databases Worth Knowing

Beyond MongoDB and Cassandra, senior backend interviews may cover these systems.

Database	Type	Main use case
Redis	Key-value	Caching, sessions, real-time state
DynamoDB	Key-value/document	Serverless scalable applications
Couchbase	Document/key-value	Distributed application data
CouchDB	Document	Replicated document storage
Neo4j	Graph	Relationships and recommendation engines
Elasticsearch	Search/document	Full-text search and log analytics
ScyllaDB	Wide-column	High-throughput, low-latency workloads

DynamoDB interview questions

What is DynamoDB?

What is a partition key?

What is a sort key?

What is a composite primary key?

What are Global Secondary Indexes (GSI)?

What are Local Secondary Indexes (LSI)?

What is eventually consistent reading?

What is strongly consistent reading?

What is DynamoDB Streams?

What is conditional writing?

What is provisioned throughput?

What is on-demand capacity?

What is a hot partition?

What is adaptive capacity?

What is TTL?

What are transactional writes?

What is the difference between DynamoDB and Cassandra?

How do you design a single-table architecture?

How do you avoid hot partitions?

How do you implement idempotency in DynamoDB?

Practical problems

Design a shopping cart using DynamoDB.

Design an order history system.

Implement conditional updates.

Implement optimistic locking.

Implement pagination using LastEvaluatedKey.

Design a multi-tenant application.

Design a high-throughput event store.

Implement DynamoDB Streams processing with Go.

9. Advanced SQL and Query Optimization

This section is important regardless of which relational database an interviewer uses.

Advanced SQL questions

What is the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN and FULL OUTER JOIN?

What is a self join?

What is a cross join?

What is a correlated subquery?

What is a CTE?

What is a recursive CTE?

What are window functions?

Difference between ROW_NUMBER(), RANK() and DENSE_RANK()?

What is PARTITION BY?

What is LEAD() and LAG()?

What is a running total?

What is a cumulative sum?

What is a lateral join?

What is query execution order?

What is a query execution plan?

What is predicate pushdown?

What is partition pruning?

What is a correlated index scan?

What is keyset pagination?

What is the difference between offset pagination and cursor pagination?

Practical SQL coding problems

No.	Problem	Level
1	Second-highest salary	Easy
2	Find duplicate emails	Easy
3	Find employees without managers	Easy
4	Find the highest salary in every department	Medium

No.	Problem	Level
5	Find top 3 employees per department	Medium
6	Find customers with no orders	Medium
7	Calculate monthly sales	Medium
8	Find consecutive login days	Medium
9	Calculate cumulative balance	Medium
10	Find the latest transaction per customer	Medium
11	Find users active for 7 consecutive days	Hard
12	Calculate rolling 7-day revenue	Hard
13	Find overlapping reservations	Hard
14	Detect fraudulent duplicate transactions	Hard
15	Calculate account balance from transaction history	Hard
16	Implement keyset pagination	Hard
17	Find missing dates in a time series	Hard
18	Calculate OHLC candles from tick data	Hard

10. Database System Design Problems

These are the most important practical problems to prepare for senior backend interviews.

1. Design a stock market tick data storage system

kdb+: The Time-Series Database of Choice for Financial Institutions | by Vikram Ranabhatt | Medium

Requirements

Ingest millions of market price updates.

Store symbol, exchange, price, quantity and timestamp.

Support historical queries.

Calculate OHLC candles.

Handle duplicate and out-of-order events.

Support retention and archival.

Discuss: Kafka, Cassandra, TimescaleDB, ClickHouse and Redis.

2. Design a high-volume order management database

Risk System Concepts - A Relational Trading Data Model | by Nik Stehr | Medium

Requirements

Store millions of orders.

Support order modification and cancellation.

Maintain order lifecycle history.

Handle concurrent updates.

Support order book and trade book queries.

Recover state after service restart.

Discuss: PostgreSQL/MySQL, indexes, transactions, Redis and Kafka.

3. Design a banking transaction database

How Software Systems Track Customer Balances

Requirements

Support deposits and withdrawals.

Maintain double-entry accounting.

Prevent duplicate transactions.

Maintain consistent account balances.

Handle concurrent transfers.

Provide an auditable transaction history.

Discuss: ACID, isolation, locking, idempotency and recovery.

4. Design a distributed logging and analytics system

Exabyte-Scale Observability: Architecting a Centralized Log Ingestion and Analytics Platform | by Dilip Kumar | Dec, 2025 | Medium

Requirements

Ingest logs from multiple services.

Support high write throughput.

Search by timestamp and service.

Aggregate metrics.

Apply retention and compression.

Support historical analysis.

Discuss: Kafka, ClickHouse, Elasticsearch and time-series storage.

11. Database practical preparation tracker

Use this checklist to track actual implementation, not just theoretical reading.

Database preparation progress

0/47

1. MySQL

Understand InnoDB internals

Implement primary and secondary indexes

Solve 15 SQL queries

Practice all isolation levels

Understand redo, undo and binary logs

Implement transactions and locking

Optimize slow queries using EXPLAIN

Understand replication and sharding

2. PostgreSQL

Understand MVCC and WAL

Study VACUUM and autovacuum

Understand B-Tree, GIN, GiST and BRIN

Practice JSONB queries

Implement transactions in Go

Optimize queries using EXPLAIN ANALYZE

Understand streaming replication

Implement keyset pagination

3. MongoDB

Practice CRUD and aggregation

Understand embedding vs referencing

Implement compound indexes

Understand replica sets

Understand sharding

Implement TTL indexes

Implement change streams using Go

4. Cassandra

Understand partition and clustering keys

Understand token ring and replication

Understand commit log and Memtable

Understand SSTables and compaction

Practice consistency levels

Design time-series partitions

Investigate tombstones and hot partitions

Implement Cassandra CRUD using Go

5. Time-series

Understand time buckets and retention

Implement OHLC candle aggregation

Implement downsampling

Understand InfluxDB

Understand TimescaleDB hypertables

Understand ClickHouse columnar storage

Handle out-of-order events

Design historical market data storage

6. Distributed database design

Design stock tick storage

Design order management database

Design banking ledger

Design distributed logging

Design multi-tenant database

Understand sharding strategies

Understand replication and failover

Design backup and recovery

Reset progress

12. Recommended learning priority

For your backend engineering preparation, I would rank the database topics as follows

Priority	Subject	Depth
1	MySQL / PostgreSQL internals	Deep
2	SQL and query optimization	Deep
3	Indexing and B+ Trees	Deep
4	Transactions, MVCC and isolation	Deep
5	Replication and sharding	Deep
6	MongoDB	Medium–Deep
7	Cassandra	Deep
8	Time-series databases	Medium–Deep
9	DynamoDB	Medium
10	ClickHouse and Elasticsearch	Medium

One important distinction: Don't learn every database using the same mental model.

MySQL/PostgreSQL: transactions, indexes, joins and consistency.

MongoDB: document modeling, aggregation and replication.

Cassandra: partition-key design, write path and tunable consistency.

Time-series: ingestion throughput, time partitioning, retention and aggregation.

ClickHouse: columnar storage and analytical query performance.

For each technology, be able to explain how data is written, how data is read, how it is indexed, how failures are recovered, and how the system scales.

I can also prepare a separate daily database interview series with SQL challenges, Go database implementation exercises, and system-design scenarios, progressing from medium to senior-level difficulty.

Integrated relational and NoSQL database engineering

SQL Fundamentals and Advanced SQL

Objective Develop strong relational database and query skills.

1. Relational database fundamentals

- Tables.
- Rows.
- Columns.
- Primary keys.

- Foreign keys.
- Unique constraints.
- Check constraints.
- Not-null constraints.
- Composite keys.
- Candidate keys.
- Referential integrity.
- Normalization.
- Denormalization.

2. SQL operations

- SELECT.
- INSERT.
- UPDATE.
- DELETE.
- UPSERT.
- MERGE.
- WHERE.
- GROUP BY.
- HAVING.
- ORDER BY.
- DISTINCT.
- LIMIT.
- OFFSET.
- CASE.
- COALESCE.
- NULL handling.

3. Joins

- INNER JOIN.
- LEFT JOIN.
- RIGHT JOIN.
- FULL OUTER JOIN.
- CROSS JOIN.
- Self join.
- Anti join.
- Semi join.
- Join cardinality.
- Join execution.

4. Advanced SQL

- Subqueries.
- Correlated subqueries.
- CTEs.
- Recursive CTEs.
- Window functions.
- ROW_NUMBER.
- RANK.
- DENSE_RANK.
- LEAD.
- LAG.
- FIRST_VALUE.
- LAST_VALUE.
- Running totals.
- Moving averages.
- Partitioning in window functions.

5. Indexing

- What is an index?
- B-tree.
- B+ tree.
- Hash indexes.
- Composite indexes.
- Covering indexes.
- Clustered versus non-clustered indexes.
- Unique indexes.
- Partial indexes.
- Functional indexes.
- Index selectivity.
- Cardinality.
- Index scan.
- Sequential scan.
- Index-only scan.
- Write amplification.

6. Query optimization

- Query execution plans.
- EXPLAIN.
- EXPLAIN ANALYZE.
- Statistics.

- Cardinality estimation.
- Join algorithms.
- Nested loop join.
- Hash join.
- Merge join.
- Sort operations.
- Query plan regression.
- N+1 queries.
- Pagination optimization.

7. SQL interview problem bank

Practice

- Second-highest salary.
- Nth-highest salary.
- Duplicate records.
- Top three salaries per department.
- Employees earning more than their managers.
- Running totals.
- Moving averages.
- Consecutive login days.
- Gaps and islands.
- Latest order per customer.
- Customers without orders.
- Duplicate financial transactions.
- Seven-day active users.
- OHLC calculations.
- Daily trading volume.

Deliverable: Solve at least 40 SQL problems and explain their execution plans.

PostgreSQL Internals and Production Engineering

1. PostgreSQL architecture

- PostgreSQL process architecture.
- Postmaster.
- Backend processes.
- Shared buffers.
- Work memory.
- WAL buffers.
- Background writer.
- Checkpointer.

- WAL writer.
- Autovacuum.
- Statistics collector concepts.
- Connection handling.

2. Storage internals

- Heap pages.
- Page headers.
- Tuple headers.
- Tuple visibility.
- TOAST.
- Free space map.
- Visibility map.
- Relation files.
- Tablespaces.
- Page layout.

3. MVCC

- Multi-Version Concurrency Control.
- Transaction IDs.
- xmin.
- xmax.
- Tuple versions.
- Snapshot visibility.
- Dead tuples.
- Vacuum.
- Autovacuum.
- Transaction ID wraparound.
- Long-running transactions.
- Bloat.

4. Transactions and isolation

- ACID.
- Atomicity.
- Consistency.
- Isolation.
- Durability.
- Read uncommitted.
- Read committed.
- Repeatable read.
- Serializable.

- Dirty reads.
- Non-repeatable reads.
- Phantom reads.
- Serialization failures.
- Retry logic.

5. Locks and concurrency

- Row locks.
- Table locks.
- Advisory locks.
- Shared locks.
- Exclusive locks.
- Deadlocks.
- Lock queues.
- Lock timeout.
- NOWAIT.
- SKIP LOCKED.
- Blocking queries.

6. WAL and durability

- Write-ahead logging.
- WAL records.
- WAL segments.
- Checkpoints.
- Commit records.
- fsync.
- Synchronous commit.
- WAL archiving.
- Point-in-time recovery.
- Crash recovery.
- Full-page writes.

7. Replication

- Streaming replication.
- Physical replication.
- Logical replication.
- Synchronous replication.
- Asynchronous replication.
- Replication slots.
- WAL sender.
- WAL receiver.

- Replication lag.
- Read replicas.
- Failover.
- Promotion.
- Split-brain risks.
- Data loss trade-offs.

8. Partitioning and operations

- Range partitioning.
- List partitioning.
- Hash partitioning.
- Partition pruning.
- Indexing partitioned tables.
- Connection pooling.
- PgBouncer.
- Backup and restore.
- pg_dump.
- pg_basebackup.
- Monitoring.
- Slow queries.
- Database migrations.

Practical assignment: Design PostgreSQL storage for orders, order updates, trades, positions and audit records.

- Include partitioning, indexing, transaction boundaries, recovery and replication.

MySQL and InnoDB

1. MySQL architecture

- MySQL server architecture.
- Connection management.
- SQL parser.
- Optimizer.
- Execution engine.
- Storage engine architecture.
- InnoDB.
- MyISAM.
- Buffer pool.
- Redo log.
- Undo log.
- Binary log.

2. InnoDB internals

- Clustered indexes.
- Secondary indexes.
- B+ tree.
- Page structure.
- Buffer pool.
- Change buffer.
- Adaptive hash index.
- Redo logging.
- Undo logging.
- Doublewrite buffer.
- MVCC.
- Purge process.

3. Transactions and locking

- Transaction isolation.
- Record locks.
- Gap locks.
- Next-key locks.
- Intention locks.
- Deadlocks.
- Lock wait timeout.
- Consistent reads.
- Current reads.

4. MySQL replication

- Binary log.
- Statement-based replication.
- Row-based replication.
- Mixed replication.
- GTID.
- Asynchronous replication.
- Semi-synchronous replication.
- Replication lag.
- Failover.
- Group Replication.
- InnoDB Cluster.

5. PostgreSQL versus MySQL

Compare

- Architecture.
- Storage.
- Indexes.
- MVCC.
- Transactions.
- Replication.
- Partitioning.
- JSON support.
- Query optimization.
- Operational complexity.
- Use cases.

Practical assignment: Implement the same order storage schema in PostgreSQL and MySQL. Compare query plans, indexes and transaction behavior.

MongoDB, Document Databases and DynamoDB

1. NoSQL fundamentals

- Why NoSQL exists.
- Relational versus NoSQL.
- Document databases.
- Key-value databases.
- Wide-column databases.
- Graph databases.
- Schema flexibility.
- Denormalization.
- Horizontal scaling.
- Partitioning.

2. MongoDB

- Documents.
- Collections.
- BSON.
- ObjectId.
- Embedded documents.
- References.
- CRUD.
- Aggregation pipeline.
- Indexes.

- Compound indexes.
- Multikey indexes.
- Text indexes.
- TTL indexes.
- Geospatial indexes.

3. MongoDB internals

- WiredTiger.
- Storage engine.
- Journaling.
- Cache.
- Document locking.
- Replication.
- Replica sets.
- Oplog.
- Primary election.
- Secondary reads.
- Read concern.
- Write concern.
- Transactions.
- Sharding.
- Chunk migration.
- Balancer.

4. DynamoDB

- Partition key.
- Sort key.
- Primary key.
- Composite key.
- Global secondary indexes.
- Local secondary indexes.
- Read capacity.
- Write capacity.
- On-demand capacity.
- Eventually consistent reads.
- Strongly consistent reads.
- Transactions.
- Conditional writes.
- DynamoDB Streams.
- TTL.

- Hot partitions.
- Adaptive capacity.

5. Data modeling

- Access-pattern-driven design.
- Single-table design.
- Denormalization.
- Read amplification.
- Write amplification.
- Hot keys.
- Partition distribution.

Practical assignment: Model an order and trade history system in MongoDB and DynamoDB. Compare access patterns and trade-offs.

Cassandra, Wide-Column Databases and Time-Series Systems

1. Cassandra fundamentals

- Distributed architecture.
- Peer-to-peer design.
- Nodes.
- Clusters.
- Data centers.
- Replication factor.
- Token ring.
- Partition key.
- Clustering columns.
- CQL.

2. Cassandra internals

- Commit log.
- Memtable.
- SSTable.
- Bloom filters.
- Compaction.
- Tombstones.
- Read path.
- Write path.
- Hinted handoff.
- Read repair.
- Anti-entropy repair.
- Gossip protocol.
- Failure detection.

3. Consistency

- ONE.
- TWO.
- QUORUM.
- LOCAL_QUORUM.
- ALL.
- Eventual consistency.
- Tunable consistency.
- Consistency versus availability.
- Lightweight transactions.
- Paxos basics.

4. Data modeling

- Query-first design.
- Partition sizing.
- Clustering order.
- Bucketing.
- Time-based partitioning.
- Avoiding hot partitions.
- Denormalization.
- Materialized views and their trade-offs.

5. Time-series databases

- Time-series data.
- Timestamp-based storage.
- Data ingestion.
- Time bucketing.
- Retention policies.
- Downsampling.
- Compression.
- Late-arriving events.
- Out-of-order events.
- High-cardinality tags.
- Aggregations.

6. Important time-series technologies

- TimescaleDB.
- InfluxDB.
- Prometheus.
- VictoriaMetrics.

- ClickHouse.
- OpenTSDB.

7. Trading-system use cases

- Market ticks.
- LTP.
- OHLC.
- Candlestick generation.
- Moving averages.
- Trading volume.
- Historical price queries.
- Time-window aggregation.
- Data retention.

Practical assignment: Build a market data ingestion pipeline that receives ticks, stores them and generates one-minute OHLC candles.

Additional database technologies and storage fundamentals

- ADDITIONAL TOPICS - DATABASES AND TECHNOLOGIES YOU SHOULD ALSO KNOW
- The following topics complement the main roadmap. Do not skip them just because they are not the primary focus of a particular week.

1. Other databases

Technology	Important concepts
DynamoDB	Partition keys, GSIs, LSIs, capacity, hot partitions
Elasticsearch	Inverted indexes, shards, replicas, analyzers, mappings
OpenSearch	Search clusters, indexing, query performance
ScyllaDB	Sharded architecture, CQL, compaction
Neo4j	Nodes, relationships, graph traversal, Cypher
Couchbase	Documents, buckets, indexes, distributed caching
ClickHouse	Columnar storage, MergeTree, compression, OLAP
InfluxDB	Time-series ingestion, retention, downsampling
TimescaleDB	Hypertables, chunks, continuous aggregates
Prometheus	Time-series metrics, labels, PromQL
VictoriaMetrics	Time-series storage and ingestion

2. Search engine fundamentals

- Inverted indexes.
- Tokenization.
- Analyzers.
- Stemming.
- Stop words.
- Relevance scoring.

- TF-IDF.
- BM25.
- Shards.
- Replicas.
- Segment merging.
- Refresh intervals.
- Near-real-time search.
- Index lifecycle management.

3. Storage fundamentals

- HDD versus SSD.
- NVMe.
- Sequential versus random I/O.
- IOPS.
- Throughput.
- Latency.
- Page cache.
- Write amplification.
- Log-structured storage.
- LSM trees.
- B-trees.
- SSTables.
- Compaction.
- Bloom filters.

4. Backend architecture patterns

Pattern	Purpose
Circuit breaker	Prevent cascading dependency failures
Bulkhead	Isolate resources and failures
Retry	Recover from transient failures
Timeout	Bound waiting time
Rate limiter	Control request rate
Backpressure	Prevent overloaded consumers
Load shedding	Reject excess work
Cache-aside	Load data into cache on demand
Write-through	Update cache and storage together
Write-behind	Persist cache updates asynchronously
Saga	Coordinate distributed business transactions
Outbox	Reliably publish database changes
CQRS	Separate read and write models
Event sourcing	Store changes as a sequence of events
Strangler fig	Gradually replace legacy systems
Sidecar	Extend a service through a companion process
Bulk processing	Process work in batches
Debouncing	Collapse repeated operations
Deduplication	Prevent repeated processing
Leader election	Select a coordinator
Fencing token	Prevent stale lock holders from acting

HLD, LLD and trading platform system design

Distributed systems, architecture and the trading-platform capstone

Distributed Systems Fundamentals

Objective Understand the fundamental problems that arise when services run across multiple machines.

1. Distributed system fundamentals

- Distributed computing.
- Network communication.
- Partial failures.
- Independent clocks.
- Network partitions.
- Latency.
- Throughput.
- Availability.
- Reliability.
- Fault tolerance.
- Scalability.

2. CAP theorem

- Consistency.
- Availability.
- Partition tolerance.
- Why partitions matter.
- CP systems.
- AP systems.
- CAP limitations.
- PACELC theorem.

3. Consistency models

- Strong consistency.
- Eventual consistency.
- Read-your-writes.
- Monotonic reads.
- Monotonic writes.
- Causal consistency.
- Linearizability.
- Sequential consistency.
- Session consistency.

4. Distributed coordination

- Leader election.
- Distributed locks.
- Leases.
- Fencing tokens.
- Quorum.
- Majority voting.
- Split brain.
- Consensus.
- Raft.
- Paxos.
- ZAB basics.

5. Replication

- Leader-follower replication.
- Multi-leader replication.
- Leaderless replication.
- Synchronous replication.
- Asynchronous replication.
- Replication lag.
- Conflict resolution.
- Read repair.
- Quorum reads and writes.

6. Distributed transactions

- Two-phase commit.
- Three-phase commit.
- Saga.
- Compensating transactions.
- Eventual consistency.
- Transaction coordinator.
- Failure recovery.

7. Distributed identifiers

- UUID.
- UUIDv4.
- UUIDv7.
- Snowflake IDs.
- Timestamp-based IDs.
- Sequence IDs.

- Collision probability.
- Clock drift.
- Clock rollback.
- Distributed ID generation.

Practical assignment: Design an order placement workflow that remains correct when PostgreSQL, Redis and Kafka fail independently.

Microservices, Design Patterns and System Architecture

1. Architecture fundamentals

- Monolith.
- Modular monolith.
- Microservices.
- Service boundaries.
- Bounded contexts.
- Domain-driven design.
- Service decomposition.
- Database per service.
- Shared database problems.
- API composition.

2. Communication

- Synchronous communication.
- Asynchronous communication.
- REST.
- gRPC.
- Message queues.
- Event streaming.
- Request-response.
- Publish-subscribe.
- Service discovery.

3. Design patterns

- Factory.
- Abstract factory.
- Builder.
- Strategy.
- Observer.
- Decorator.
- Adapter.
- Facade.
- Proxy.

- Singleton.
- Repository.
- Unit of Work.
- Dependency Injection.
- Chain of Responsibility.
- Template Method.
- Command.
- State.

4. Distributed architecture patterns

- API Gateway.
- Backend for Frontend.
- Service registry.
- Service mesh.
- Sidecar.
- Saga.
- CQRS.
- Event sourcing.
- Transactional outbox.
- Transactional inbox.
- Strangler fig.
- Anti-corruption layer.
- Bulkhead.
- Circuit breaker.

5. Data consistency patterns

- Database-per-service.
- Eventual consistency.
- Materialized views.
- Read models.
- Write models.
- Change Data Capture.
- Event replay.
- Idempotency.
- Deduplication.

6. Versioning and compatibility

- API versioning.
- Event versioning.
- Schema evolution.
- Expand-and-contract migrations.

- Backward compatibility.
- Forward compatibility.
- Rolling upgrades.

Practical assignment: Split your order platform into

- API Gateway.
- Order Service.
- Risk Service.
- Account Service.
- OMS Adapter.
- Market Data Service.
- Reporting Service.
- Notification Service.
- Define the responsibilities and communication patterns of each.

Cloud Computing, AWS and Backend Security

1. Cloud fundamentals

- IaaS.
- PaaS.
- SaaS.
- Public cloud.
- Private cloud.
- Hybrid cloud.
- Regions.
- Availability zones.
- Edge locations.
- Shared responsibility model.

2. AWS fundamentals

- EC2.
- ECS.
- EKS.
- Lambda.
- S3.
- RDS.
- Aurora.
- DynamoDB.
- ElastiCache.
- MSK.
- SQS.

- SNS.
- EventBridge.
- API Gateway.
- CloudFront.
- Route 53.
- IAM.
- CloudWatch.
- CloudTrail.
- Secrets Manager.
- Systems Manager.

3. Networking in AWS

- VPC.
- Subnets.
- Route tables.
- Internet Gateway.
- NAT Gateway.
- Security Groups.
- Network ACL.
- VPC peering.
- Transit Gateway.
- PrivateLink.
- VPC endpoints.
- Public versus private subnets.

4. Identity and security

- Authentication.
- Authorization.
- RBAC.
- ABAC.
- IAM roles.
- IAM policies.
- Least privilege.
- OAuth 2.0.
- OpenID Connect.
- JWT.
- Access tokens.
- Refresh tokens.
- API keys.
- mTLS.

- Secret rotation.

5. Application security

- OWASP Top 10.
- SQL injection.
- XSS.
- CSRF.
- SSRF.
- Broken access control.
- Authentication failures.
- Sensitive data exposure.
- Secure headers.
- Input validation.
- Output encoding.
- Password hashing.
- TLS.
- Encryption at rest.
- Encryption in transit.

6. CI/CD

- Continuous integration.
- Continuous delivery.
- Continuous deployment.
- Build pipelines.
- Unit tests.
- Integration tests.
- Contract tests.
- Static analysis.
- Dependency scanning.
- Container scanning.
- Artifact repositories.
- Deployment strategies.
- Rollback strategies.
- Infrastructure as Code.
- Terraform.
- Helm.

Practical assignment: Build a CI/CD pipeline that tests, builds, scans and deploys your Go order service.

Monitoring, Logging, Tracing and Production Reliability

1. Observability fundamentals

Understand the three pillars

- Metrics.
- Logs.
- Traces.
- Monitoring versus observability.
- White-box monitoring.
- Black-box monitoring.
- Golden signals.
- RED method.
- USE method.
- Correlation IDs.
- Trace IDs.
- Span IDs.

2. Logging

- Structured logging.
- JSON logs.
- Log levels.
- Log sampling.
- Correlation IDs.
- Sensitive data masking.
- Centralized logging.
- Log retention.
- Log aggregation.

3. Metrics

- Counter.
- Gauge.
- Histogram.
- Summary.
- Percentiles.
- P50.
- P90.
- P95.
- P99.
- Throughput.
- Error rate.

- Saturation.
- Cardinality.

4. Distributed tracing

- OpenTelemetry.
- Trace context propagation.
- Spans.
- Parent-child spans.
- Sampling.
- Head-based sampling.
- Tail-based sampling.
- Distributed trace visualization.

5. Monitoring tools

- Prometheus.
- Grafana.
- Alertmanager.
- Loki.
- Elasticsearch.
- Kibana.
- OpenTelemetry Collector.
- Jaeger.
- Tempo.
- Datadog.
- Dynatrace.

6. SRE concepts

- Service Level Indicator.
- Service Level Objective.
- Service Level Agreement.
- Error budgets.
- Availability.
- Reliability.
- Incident management.
- On-call.
- Postmortems.
- Toil.
- Capacity planning.
- Disaster recovery.
- RTO.
- RPO.

7. Chaos engineering

- Fault injection.
- Dependency failures.
- Network latency injection.
- Packet loss.
- DNS failure.
- CPU exhaustion.
- Memory exhaustion.
- Disk pressure.
- Pod termination.
- Broker failure.
- Database failover.
- Redis failure.
- Consumer lag.
- Recovery validation.

Run chaos tests against the order platform

- Kill a Kafka broker.
- Stop PostgreSQL.
- Restart Redis.
- Introduce network latency.
- Kill a consumer.
- Exhaust a connection pool.
- Introduce high request volume.
- Record detection time, failure behavior, recovery time and data consistency.

Low-Level Design and High-Level Design

Objective Develop the ability to design complete production systems and explain their trade-offs.

1. Low-Level Design

- Object-oriented design.
- SOLID principles.
- Encapsulation.
- Abstraction.
- Polymorphism.
- Composition.
- Interfaces.
- Class relationships.
- UML.
- Sequence diagrams.

- State diagrams.
- Design patterns.
- Testability.
- Extensibility.

Practice designing

- Parking lot.
- Elevator.
- Payment system.
- Notification system.
- Rate limiter.
- LRU cache.
- Splitwise.
- Library management.
- Order management.
- Trading rules engine.

2. High-Level Design fundamentals

- Functional requirements.
- Non-functional requirements.
- Capacity estimation.
- QPS.
- TPS.
- Storage estimation.
- Bandwidth estimation.
- Latency budgets.
- Availability targets.
- Scalability.
- Reliability.
- Cost trade-offs.

3. Common system components

- API Gateway.
- Load Balancer.
- CDN.
- Reverse Proxy.
- Application servers.
- SQL databases.
- NoSQL databases.
- Redis.
- Kafka.

- Object storage.
- Search engines.
- Background workers.
- Scheduler.
- Notification systems.

4. Scaling

- Vertical scaling.
- Horizontal scaling.
- Stateless services.
- Stateful services.
- Database read replicas.
- Sharding.
- Partitioning.
- Caching.
- Queue-based load leveling.
- Asynchronous processing.
- Database connection pooling.

5. System design problems

Design

- URL Shortener.
- Pastebin.
- WhatsApp.
- Twitter/X.
- Instagram.
- YouTube.
- Netflix.
- Uber.
- Notification system.
- Rate limiter.
- Distributed job scheduler.
- Payment gateway.
- Stock trading platform.
- Order management system.
- Real-time market data platform.

6. Specific design: Pastebin

- Unique ID generation.
- Snowflake IDs.

- Paste creation.
- Paste retrieval.
- Expiration.
- Metadata storage.
- Text storage.
- Redis caching.
- CDN caching.
- Read-heavy workloads.
- Abuse prevention.
- Rate limiting.
- Data retention.
- Sharding.

7. Specific design: URL shortener

- Short code generation.
- Base62 encoding.
- Snowflake ID.
- ID collision.
- Redirect flow.
- Cache-aside.
- Read replicas.
- Analytics.
- Expiration.
- Custom aliases.
- Hot URLs.
- Database partitioning.

Build a Distributed Order Management Platform

Objective Combine the entire roadmap into one production-style backend project.

- Capstone: Distributed Trading Order Platform
- tldraw-skill - From Text to Whiteboard Diagrams
- agents365-ai.github.io

Technology stack

- Go for backend services.
- PostgreSQL for durable order storage.
- Redis for real-time order state.
- Kafka for event streaming.
- Docker for containers.
- Kubernetes for orchestration.
- Prometheus and Grafana for metrics.

- OpenTelemetry for tracing.
- AWS for cloud deployment.

1. Services to build

- API Gateway
- Authentication.
- Request validation.
- Rate limiting.
- Routing.
- Correlation IDs.
- Order Service
- Place order.
- Modify order.
- Cancel order.
- Get order.
- Order validation.
- Idempotency.
- Risk Service
- Margin validation.
- Funds validation.
- Position limits.
- Trading restrictions.
- OMS Adapter
- Submit orders to external OMS.
- Handle acknowledgements.
- Handle rejections.
- Retry failed submissions.
- Track order lifecycle.
- Market Data Service
- Consume market ticks.
- Maintain LTP.
- Generate OHLC.
- Publish market events.
- Redis State Updater
- Maintain orderbook.
- Maintain positions.
- Maintain holdings.
- Maintain trades.
- Recover state after restart.

- Reporting Service
- Generate order reports.
- Generate trade reports.
- Generate position reports.
- Support historical queries.

2. Database schema

Design tables for

- orders
- order_updates
- trades
- positions
- holdings
- order_events
- outbox_events
- idempotency_keys
- audit_logs

Define

- Primary keys.
- Foreign keys.
- Unique constraints.
- Indexes.
- Partitioning.
- Transactions.
- Retention.

3. Event architecture

Example

- Client
- |
- v
- API Gateway
- |
- v
- Order Service
- |
- +----> Risk Service
- |
- v

- PostgreSQL Transaction
- |
- +----> Orders
- |
- +----> Outbox Events
- |
- v
- Kafka Publisher
- |
- v
- Kafka
- |
- +-------+--------+
- | |
- v v
- OMS Adapter Redis State Updater
- | |
- v v
- External OMS Redis Orderbook

4. Failure scenarios

Test all of the following

- API request is duplicated.
- PostgreSQL becomes unavailable.
- Kafka becomes unavailable.
- Redis restarts.
- Kafka consumer crashes.
- OMS times out.
- OMS rejects an order.
- Network connection breaks.
- Redis contains stale state.
- Consumer processes duplicate events.
- Two requests modify the same order.
- Database transaction commits but Kafka publishing fails.
- Kafka publishing succeeds but consumer processing fails.
- One Kubernetes pod crashes.
- One availability zone becomes unavailable.

5. Non-functional requirements

Define and measure

- Throughput.
- P50 latency.
- P95 latency.
- P99 latency.
- Error rate.
- Availability.
- Data durability.
- Recovery time.
- Horizontal scalability.
- Backpressure.
- Idempotency.
- Ordering guarantees.

6. Final deliverables

- Working Go microservices.
- PostgreSQL schema and migrations.
- Redis data model.
- Kafka event architecture.
- Docker Compose setup.
- Kubernetes deployment manifests.
- Unit tests.
- Integration tests.
- Load tests.
- Chaos tests.
- Metrics dashboards.
- Distributed tracing.
- Architecture diagrams.
- System design documentation.
- Failure recovery documentation.

Weekly practice structure

A practical daily schedule for approximately 3 hours

Activity	Time
Learn theory and internals	60 min
Read documentation and source-level explanations	30 min
Implement a practical example	60 min
Solve interview questions	20 min
Write notes and review	10 min

How to study each topic

For every technology, answer these questions

- What problem does it solve?
- Why was it introduced?
- How does it work internally?
- What are its major components?
- What are its limitations?
- What happens when it fails?
- How does it scale?
- What are the alternatives?
- When should you use it?
- What trade-offs does it introduce?
- How would you monitor it in production?
- How would you debug it during an incident?

Recommended depth by topic

- Suggested allocation of study effort, not an assessment of topic importance.
- Suggested study-hour distribution across the most important backend engineering areas.

Senior backend readiness checklist

- Use this as your progress tracker.

1. Go and programming

- Go language internals
- Interfaces and dependency injection
- Goroutines and channels
- Go scheduler G-M-P model
- Memory management and garbage collection
- Profiling and performance optimization
- Testing and race detection

2. Networking and resilience

- TCP/IP and DNS
- HTTP/1.1, HTTP/2 and HTTP/3
- TLS and HTTPS
- Reverse proxy and load balancing
- CDN and edge caching
- Circuit breaker and retry
- Timeout, bulkhead and backpressure

3. Infrastructure

- Docker internals
- Linux namespaces and cgroups
- Kubernetes architecture
- Deployments and StatefulSets
- Autoscaling and resource management

4. Redis and Kafka

- Redis data structures and internals
- Redis persistence and replication
- Redis Sentinel and Cluster
- Caching strategies and cache failures
- Kafka architecture and replication
- Kafka producer and consumer internals
- Kafka delivery semantics
- Outbox, retries and DLQ

5. Databases

- SQL and advanced queries
- Indexes and query optimization
- PostgreSQL MVCC and WAL
- PostgreSQL replication and partitioning
- MySQL and InnoDB internals
- MongoDB and DynamoDB
- Cassandra internals
- Time-series databases

6. Distributed systems and architecture

- CAP and consistency models
- Consensus and leader election
- Distributed transactions and Saga
- Microservices architecture

- CQRS and event sourcing
- Low-level design
- High-level system design

7. Production engineering

- AWS and cloud networking
- Authentication and authorization
- CI/CD and deployment strategies
- Metrics, logs and traces
- SLO, SLA and error budgets
- Chaos engineering
- Disaster recovery and capacity planning

Do not study these topics only as definitions. For every major concept, implement something, break it intentionally, observe the failure and fix it.

For a senior backend engineer, the most important areas to master deeply are

- Go concurrency, runtime and memory management.
- PostgreSQL internals, indexing, MVCC and transactions.
- Redis internals, replication, caching and recovery.
- Kafka delivery guarantees, partitioning and consumer recovery.
- Networking, CDN, load balancing and resilience.
- Distributed systems consistency and failure handling.
- Low-level and high-level system design.

The final objective: Be able to design, implement, scale, monitor and debug a distributed backend system-not just explain what each technology does.

HLD interview framework

* Clarify functional requirements, users, traffic patterns and non-functional targets: p95/p99 latency, throughput, availability, durability, recovery and cost.
* Estimate QPS/TPS, peak-to-average load, storage growth, bandwidth, queue drain time and cache working set. State assumptions and show the arithmetic.
* Define APIs, event contracts, entity ownership and data models before choosing components. Draw read and write paths and explain synchronous versus asynchronous boundaries.
* Choose databases, indexes, partitioning, replication, caching, CDN, queues and consistency guarantees based on workload. Identify ordering, deduplication and idempotency boundaries.
* Walk through hot keys, overload, dependency outage, regional failure, data loss, replay, backup restoration, security, privacy and operational visibility.
* Discuss alternatives and tradeoffs, bottlenecks, evolution plan, rollout, migration and rollback. Avoid claiming exactly-once effects without describing transactional boundaries.

LLD interview framework

* Model domain objects, invariants, state transitions, value types and ownership. Keep interfaces small and put behavior with the data it protects.
* Apply SOLID and composition where they improve changeability; explain when a pattern adds needless indirection. Design for dependency injection and deterministic tests.

* Define public methods, error types, concurrency controls, idempotency, lifecycle and resource cleanup. Cover validation, retries and cancellation.

* Use sequence/state diagrams or pseudocode to show interactions. Include edge cases, complexity, extensibility, persistence boundaries and test cases.

High-yield design problems

* LLD: parking lot, elevator, payment gateway, notification dispatcher, library, Splitwise, LRU cache, token-bucket rate limiter, job scheduler, order lifecycle/state machine, order book and trading rules engine.

* HLD: URL shortener, Pastebin, API rate limiter, distributed ID generator, notification service, chat/feed, video delivery, search/log analytics, distributed cache, job scheduler, payment and double-entry ledger, inventory, market-data ingestion, real-time leaderboard, order-management/trading platform.

* For every design, practice requirements clarification, APIs, data model, component diagram, request/event flow, capacity estimates, consistency, failure recovery, security, monitoring, tradeoffs and a phased rollout.